



SAYMAN Blockchain

Complete Technical Documentation — Phase 22

JavaScript-native Smart Contracts · Proof-of-Stake · Multi-Layer
Chains · EVM Compatible

PHASE 22

PUBLIC TESTNET

CHAIN ID 82922

METAMASK READY

TSAYN

MIT LICENSE

RPC ENDPOINT

sayman.onrender.com

EXPLORER

sayman.up.railway.app

TICKER (TESTNET)

tSAYN

BLOCK TIME

5 seconds

DECIMALS

8 (sprinkles)

FAUCET

t.me/SaymanLa1



Table of Contents

01	Overview & What's New in Phase 22	README.md
02	Network Info & Wallet Connection (MetaMask, Trust Wallet, all EVM wallets)	NETWORK_INFO.md
03	Installation & Quick Start Guide	INSTALL.md
04	Operating a Multi-Peer Testnet	summary.md
05	Blockchain Comparison & Phase-by-Phase Roadmap (Phase 1-22)	comparison.md
06	Pitch Deck — Executive Summary & Go-To-Market Strategy	pitch.md
07	Pull Request — Phase 22 Changes & Testing Checklist	PULL_REQUEST.md
08	Developer Reference & Project Memory (AI.md)	AI.md
09	About SAYMAN — The Complete Story	ABOUT.md

SAYMAN BLOCKCHAIN — PHASE 22: FULL EVM/METAMASK COMPAT

• tSAYN • NONCE MANAGER • EXPLORER 2.0

JavaScript-native Smart Contracts · Proof-of-Stake · Multi-Layer Chains · Custom Tokens · NFTs · DEX · Memecoins · Staking Pools

Phase **22** Network **Public Testnet** License **MIT**

What's New in Phase 22

Phase 22 delivers **full MetaMask & EVM wallet compatibility**, the **tSAYN testnet symbol** (following Ethereum convention), an **atomic NonceManager** that eliminates nonce race conditions, **Explorer 2.0** with dedicated Token/NFT/Memecoin/Address pages, and a unified search bar across all entity types.

Feature	Status
Full MetaMask Compatibility	✓ <code>eth_getLogs</code> , <code>eth_newBlockFilter</code> , <code>eth_getFilterChanges</code> , <code>wallet_addEthereumChain</code> , 0x address stripping, correct balance scale ($\times 10^{10}$), nonce pending-state tracking
tSAYN Testnet Symbol	✓ Testnet API returns <code>tSAYN</code> ; mainnet returns <code>SAYN</code> — follows Ethereum convention (SepoliaETH, Mumbai MATIC)
Atomic Nonce Manager	✓ <code>core/nonce.js</code> — per-address in-memory nonce tracking with <code>getNonce/commitNonce/rollbackNonce</code> ; <code>/api/nonce/:addr</code> endpoint; wallet fetches fresh nonce before every broadcast
Explorer 2.0 Pages	✓ Dedicated explorer pages: Tokens · NFTs · Memecoins · unified Address (balance + txs + tokens + NFTs + reputation in one view)
Unified Search	✓ Search bar resolves block index, tx hash, address, token name/symbol from a single input
Live L2 Layer Status	✓ <code>/layers</code> page shows real-time chain status for each registered L2/sidechain

Feature	Status
Gas Fee Accuracy	✅ Explorer now shows "Fee Paid" in tSAYN (not raw gas units)
Contract State & ABI Fix	✅ <code>_extractABI</code> correctly handles <code>methods:{...}</code> style contracts; state viewer rendered properly
SAYMAN Logo Asset	✅ 512×512 PNG served at <code>/assets/Logo-512.png</code> for MetaMask network icon recognition
PUKY Wallet APK v22	✅ New signed APK — tSAYN display, fresh nonce on every broadcast, <code>addSaymanToMetaMask()</code> one-click function



MetaMask & Global Wallet Integration

SAYMAN supports standard **Ethereum JSON-RPC 2.0**. Connect MetaMask, Trust Wallet, Coinbase Wallet, or any EVM-compatible wallet using:

Network Connection Settings

Parameter	Public Testnet	Local Development
Network Name	Sayman Public Testnet	Sayman Local Testnet
New RPC URL	<code>https://sayman.onrender.com</code>	<code>http://localhost:10000</code>
Chain ID	82922 (hex <code>0x143ea</code>)	82923 (hex <code>0x143eb</code>)
Currency Symbol	tSAYN	tSAYN
Block Explorer	<code>https://sayman.up.railway.app</code>	<code>http://localhost:10000</code>
Logo	<code>https://sayman.onrender.com/assets/logo-512.png</code>	—

One-click setup: Visit <https://sayman.up.railway.app> → Network page → "Add to MetaMask" button.

Supported JSON-RPC Methods (Phase 22)

Method	Description
<code>eth_chainId</code> / <code>net_version</code>	Returns <code>82922</code> (testnet)
<code>eth_blockNumber</code>	Latest block height in hex
<code>eth_getBlockByNumber</code> / <code>eth_getBlockByHash</code>	Block data in EVM format
<code>eth_getBalance</code>	Balance in Wei-equivalent (base units $\times 10^{10}$)
<code>eth_getTransactionCount</code>	Account nonce (confirmed + pending)
<code>eth_sendRawTransaction</code>	EIP-155 / EIP-2930 / EIP-1559 tx, ECDSA recovery → SAYMAN mempool
<code>eth_getTransactionByHash</code> / <code>eth_getTransactionReceipt</code>	Tx details + receipt with <code>effectiveGasPrice</code>
<code>eth_estimateGas</code>	Returns <code>0x5208</code> (21000)
<code>eth_gasPrice</code>	Gas price in Wei-equivalent
<code>eth_getLogs</code>	Returns <code>[]</code> (SAYMAN events not ABI-encoded)
<code>eth_newBlockFilter</code> / <code>eth_getFilterChanges</code>	Block filter poll — returns latest block hash
<code>eth_newPendingTransactionFilter</code> / <code>eth_uninstallFilter</code>	Filter lifecycle
<code>wallet_addEthereumChain</code> / <code>wallet_switchEthereumChain</code>	EIP-3085 handled — <code>null</code> = success
<code>eth_accounts</code> / <code>eth_requestAccounts</code>	Returns <code>[]</code> (non-custodial)
<code>eth_syncing</code>	<code>false</code>
<code>net_listening</code>	<code>true</code>
<code>net_peerCount</code>	Live peer count
<code>web3_clientVersion</code>	SAYMAN/v22.0.0/javascript



Architecture

```
sayman/
├─ core/
│   ├── blockchain.js      # PoS engine, NonceManager integration
│   ├── nonce.js           # NEW Phase 22: Atomic per-address NonceManager
│   └─ chain-factory.js    # ChainFactory: L2/Sidechain/Permissioned config
├─ builder
│   ├── rollup.js          # L2 state root commitment to L1
│   ├── state.js           # StateEngine: balances, stakes, reputation, contracts
│   ├── contracts.js       # Sandboxed JS VM contract execution
│   ├── gas.js             # Gas calculator
│   └─ ...
├─ contracts/
│   ├── token.js           # ERC-20 style token
│   ├── nft.js             # ERC-721 style NFT
│   ├── token-factory.js   # Token factory
│   ├── nft-factory.js     # NFT collection factory
│   ├── memecoin-factory.js # Memecoin launcher (burn/tax/anti-whale)
│   ├── dex.js             # AMM DEX (Uniswap V2 style)
│   ├── staking-pool.js    # Delegated staking pool
│   └─ layer2Bridge.js      # L1 bridge for L2 deposits/withdrawals
├─ p2p/server.js           # P2P sync + peer reputation points
├─ api/routes.js           # REST API – tokens, NFTs, memecoins, address, nonce
├─ endpoints
├─ assets/                 # NEW Phase 22: logo-512.png served for MetaMask
├─ frontend/               # Explorer 2.0 – Tokens/NFTs/Memecoins/Address pages,
├─ unified search
├─ config/                 # Network configs (testnet, public-testnet, mainnet)
└─ server.js               # Entry point – EVM RPC handler + static asset serving
```



Tokenomics & Denomination

Property	Testnet	Mainnet
Ticker	tSAYN	SAYN
Base Unit	sprinkle (1 SAYN = 100,000,000 sprinkles)	same
Decimals	8	8
Block Time	5 seconds	5 seconds
Block Reward	0.5 SAYN = 50,000,000 sprinkles	0.2 SAYN
Min Stake	10 SAYN = 1,000,000,000 sprinkles	500 SAYN

Property	Testnet	Mainnet
Max Supply	Unlimited (testnet)	100,000,000 SAYN

API Clarity: All on-chain amounts are integers in base units (sprinkles). Call `GET /api/denomination` for the conversion table.

Multi-Layer Chains

```
import { ChainFactory } from './core/chain-factory.js';

// Layer 2 Rollup (anchored to SAYMAN L1)
const config = ChainFactory.createL2Config({
  name: 'MyL2Chain',
  chainId: 'my-l2-1',
  l1RpcUrl: 'https://sayman.onrender.com',
  l1Bridge: '<BRIDGE_CONTRACT_ADDRESS>',
  genesis: { '<YOUR_ADDRESS>': 1_000_000_000 }
});

// Independent Sidechain
const sideConfig = ChainFactory.createSidechainConfig({
  name: 'GameChain',
  chainId: 'gamechain-1',
  blockReward: 10_000_000,
});

// Permissioned (private consortium)
const permConfig = ChainFactory.createPermissionedConfig({
  name: 'EnterpriseChain',
  chainId: 'ent-1',
  validators: ['<ADDR1>', '<ADDR2>']
});
```

Running Locally

```
git clone https://github.com/saymanlal/SAYMAN
cd SAYMAN
npm install

# Start validator node
node server.js --network testnet --mode validator
```

```
# Sync-only observer
node server.js --network testnet --mode observer
```

Environment Variables

```
PORT=10000
NETWORK=public-testnet
MODE=validator
BOOTSTRAP_PEERS=wss://peer1.example.com/p2p
DB_PATH=./data/node-10000
```

Explorer

Open `http://localhost:10000` . Explorer 2.0 includes:

-  Dashboard — TPS, denomination, APR, mempool
-  Block & transaction explorer with unified search
-  Validators — stake (tSAYN), reputation, missed blocks
-  Tokens / NFTs / Memecoins — factory-deployed assets
-  Address — balance + tx history + tokens + NFTs + reputation
-  Layers — L2/sidechain live status
-  Network — peers, node info, MetaMask one-click add

REST API

```
# Network info (layer, decimals, denomination, ticker)
curl https://sayman.onrender.com/api/network

# SAYN conversion guide
curl https://sayman.onrender.com/api/denomination

# Fresh nonce for address (use before broadcast)
curl https://sayman.onrender.com/api/nonce/<address>

# Custom tokens list
curl https://sayman.onrender.com/api/tokens

# NFT collections
curl https://sayman.onrender.com/api/nfts

# Memecoins
curl https://sayman.onrender.com/api/memecoins

# Unified address view (balance + txs + tokens + NFTs + reputation)
curl https://sayman.onrender.com/api/address/<address>
```



```
# Submit transaction
curl -X POST https://sayman.onrender.com/api/broadcast \
-H "Content-Type: application/json" \
-d '{"type":"TRANSFER","data":{"..."},"signature":"...","publicKey":"..."}'
```



Smart Contracts

```
// Deploy a custom token:
client.callContract({
  contractAddress: TOKEN_FACTORY_ADDR,
  method: 'createToken',
  args: { name: 'DOGE ON SAYMAN', symbol: 'SDOGE', totalSupply: 1_000_000_000 }
});

// Launch a memecoin:
client.callContract({
  contractAddress: MEMECOIN_FACTORY_ADDR,
  method: 'launch',
  args: {
    name: 'PepeSAYN', symbol: 'PSAYN', totalSupply: 420_000_000_000,
    transferTaxPercent: 2, maxWalletPercent: 1, burnOnTransfer: true
  }
});

// Add liquidity to DEX:
client.callContract({
  contractAddress: DEX_ADDR,
  method: 'addLiquidity',
  args: { tokenA: 'ADDR_A', tokenB: 'ADDR_B', amountA: 1000, amountB: 2000 }
});
```



PUKY Mobile Wallet (v22)

Android APK at [/apk/puky.apk](#) (also mirrored at github.com/saymanlal/puky). Features:

- MetaMask-compatible: `addSaymanToMetaMask()` one-click network setup
- Fresh nonce fetched before every broadcast (no nonce race conditions)
- Balance displayed in **tSAYN** (testnet) / **SAYN** (mainnet)
- Multi-RPC failover (Railway + Render + local)
- Send / receive / stake / unstake



Contributing

See `ABOUT.md` for full project vision. PRs welcome on `main` branch.

Phase 22 Updates

- **Full MetaMask EVM Compat:** `eth_getLogs` , `eth_newBlockFilter` , `eth_getFilterChanges` , `wallet_addEthereumChain` , 0x address auto-stripping, balance scale fix, pending nonce tracking.
 - **tSAYN Testnet Symbol:** API, explorer, wallet all show `tSAYN` on testnet; `SAYN` on mainnet.
 - **Atomic NonceManager:** `core/nonce.js` — eliminates nonce race conditions; `/api/nonce/:addr` endpoint; wallet uses fresh nonce before every broadcast.
 - **Explorer 2.0:** Tokens, NFTs, Memecoins, unified Address pages; unified search; live L2 layer status; accurate gas fee in tSAYN; contract state & ABI viewer fix.
 - **PUKY APK v22:** New signed build with all Phase 22 features.
-



License

MIT — see `LICENSE` file.



Links

- **Explorer:** <https://sayman.up.railway.app>
- **RPC:** <https://sayman.onrender.com>
- **Wallet (web):** <https://sayman-wallet-manager.vercel.app>
- **Faucet:** <https://sayman-faucet-site.vercel.app>
- **Docs:** <https://sayman-docs.vercel.app>
- **GitHub:** <https://github.com/saymanlal/SAYMAN>
- **Telegram (bulk tokens):** <https://t.me/SaymanLal>
- **IP Owner:** [Vizkus Groups](#) (Cybokrafts Universal Innovations Pvt. Ltd.)

SAYMAN Blockchain — Network Info & Wallet Connection Guide

Phase 22 | Public Testnet | Chain ID: 82922

RPC Endpoint: <https://sayman.onrender.com>

Block Explorer: <https://sayman.up.railway.app>

GitHub: <https://github.com/saymanlal/SAYMAN>



Connect Your Wallet (MetaMask & All Global Wallets)

SAYMAN supports the standard **Ethereum JSON-RPC 2.0** protocol. Any wallet that accepts custom networks (MetaMask, Trust Wallet, Coinbase Wallet, Rainbow, Rabby, imToken, Exodus, Brave Wallet, etc.) can connect using the parameters below.

Public Testnet (Live Network)

Parameter	Value
Network Name	Sayman Public Testnet
New RPC URL	https://sayman.onrender.com
Chain ID	82922
Currency Symbol	tSAYN
Block Explorer URL	https://sayman.up.railway.app
Network Hex ID	0x143ea
Logo	https://sayman.onrender.com/assets/logo-512.png

Local Development Node

Parameter	Value
Network Name	Sayman Local Testnet
New RPC URL	http://localhost:10000

Parameter	Value
Chain ID	82923
Currency Symbol	tSAYN
Block Explorer URL	http://localhost:10000

⚡ **One-click setup:** Visit <https://sayman.up.railway.app> → Network page → "Add to MetaMask" button (EIP-3085 `wallet_addEthereumChain`).



Step-by-Step: Add SAYMAN to MetaMask

Desktop (MetaMask Browser Extension)

1. Open MetaMask → click the network dropdown at the top
2. Click "Add network"
3. Click "Add a network manually" at the bottom
4. Fill in the fields:
 - **Network name:** Sayman Public Testnet
 - **New RPC URL:** <https://sayman.onrender.com>
 - **Chain ID:** 82922
 - **Currency symbol:** tSAYN
 - **Block explorer URL:** <https://sayman.up.railway.app>
5. Click **Save** → then "Switch to Sayman Public Testnet"

Mobile (MetaMask App)

1. Open MetaMask → tap the hamburger menu
2. Tap **Settings** → **Networks** → **Add Network**
3. Enter the same values as above (use tSAYN as symbol)
4. Tap **Add** to confirm



Add to Other Wallets

Trust Wallet

1. Go to Settings → Preferences → Custom Network

2. Enter: RPC URL `https://sayman.onrender.com` , Chain ID `82922` , Symbol `tSAYN`

Coinbase Wallet

1. Settings → Active Networks → Add Custom Network
2. Enter the same network details as above

Rabby / Rainbow / Brave Wallet

- All support EIP-3085 (`wallet_addEthereumChain`) — use the same values above

WalletConnect

- SAYMAN is EVM JSON-RPC 2.0 compatible; use Chain ID `82922` to connect



Supported JSON-RPC Methods (Phase 22)

Method	Description
<code>eth_chainId</code>	Returns <code>0x143ea</code> (82922 for public testnet)
<code>net_version</code>	Returns <code>"82922"</code>
<code>eth_blockNumber</code>	Latest block height in hex
<code>eth_getBlockByNumber</code>	Block data (EVM-format) by number
<code>eth_getBlockByHash</code>	Block data (EVM-format) by hash
<code>eth_getBalance</code>	Address balance in Wei-equivalent (base units × 10 ¹⁰)
<code>eth_getTransactionCount</code>	Account nonce (confirmed + pending max)
<code>eth_sendRawTransaction</code>	Submit signed EIP-155/2930/1559 transaction
<code>eth_getTransactionByHash</code>	Transaction details by hash
<code>eth_getTransactionReceipt</code>	Transaction receipt (status, gas used, <code>effectiveGasPrice</code>)
<code>eth_estimateGas</code>	Gas estimate (returns <code>0x5208</code> = 21000)
<code>eth_gasPrice</code>	Current gas price in Wei-equivalent
<code>eth_getLogs</code>	Returns <code>[]</code> (SAYMAN events are not EVM ABI-encoded)

Method	Description
<code>eth_newBlockFilter</code>	Creates filter; returns <code>0x1</code>
<code>eth_getFilterChanges</code>	Returns latest block hash for liveness polling
<code>eth_newPendingTransactionFilter</code>	Returns <code>0x2</code>
<code>eth_newFilter</code>	Returns <code>0x3</code>
<code>eth_getFilterLogs</code>	Returns <code>[]</code>
<code>eth_uninstallFilter</code>	Returns <code>true</code>
<code>eth_accounts</code>	Returns <code>[]</code> (non-custodial)
<code>eth_requestAccounts</code>	Returns <code>[]</code>
<code>wallet_addEthereumChain</code>	EIP-3085 — acknowledged, returns <code>null</code>
<code>wallet_switchEthereumChain</code>	Returns <code>null</code>
<code>wallet_getPermissions</code>	Returns <code>[]</code>
<code>eth_getCode</code>	Returns <code>0x</code>
<code>eth_syncing</code>	<code>false</code>
<code>net_listening</code>	<code>true</code>
<code>net_peerCount</code>	Live peer count in hex
<code>web3_clientVersion</code>	<code>SAYMAN/v22.0.0/javascript</code>



Example: curl JSON-RPC Calls

```
# Get chain ID
curl -X POST https://sayman.onrender.com \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"eth_chainId","params":[],"id":1}'
# → {"result":"0x143ea"}

# Get latest block number
curl -X POST https://sayman.onrender.com \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}'
```

```
# Get balance (0x-prefixed MetaMask address works)
curl -X POST https://sayman.onrender.com \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"eth_getBalance","params":
["0xYOUR_ADDRESS","latest"],"id":1}'

# Get nonce (fresh – use before every broadcast to avoid conflicts)
curl https://sayman.onrender.com/api/nonce/YOUR_ADDRESS
```



Network Overview (Phase 22)

Parameter	Value
Network Name	Sayman Public Testnet
Chain ID (string)	sayman-public-testnet-1
Chain ID (numeric)	82922
Layer	L1 (Base chain)
Block Time	5 seconds
Block Reward	0.5 tSAYN (50,000,000 base units)
Min Stake	10 tSAYN (1,000,000,000 base units)
Max Supply (Testnet)	Unlimited
Max Supply (Mainnet)	100,000,000 SAYN
Ticker (Testnet)	tSAYN
Ticker (Mainnet)	SAYN
Base Unit	sprinkle (1 SAYN = 100,000,000 sprinkles)
Decimals	8
Gas Price	1 base unit per gas unit
Transfer Gas	21,000 gas
Faucet	☒ Enabled
Faucet Amount	1,000 tSAYN per drip

Parameter	Value
Faucet Cooldown	10 minutes
Bulk Tokens	Telegram @SaymanLal (up to 10M tSAYN for builders)



Gas Model

Gas Limits

Parameter	Value
Min Gas Price	1 wei
Max Gas Per Transaction	10,000,000
Max Gas Per Block	100,000,000
Max Execution Time	5,000ms
Max State Size	500KB
Max Instructions	100,000

Gas Costs

Transaction Type	Base Gas Cost
TRANSFER	21,000
STAKE	50,000
UNSTAKE	50,000
CONTRACT_DEPLOY	200,000 + code size
CONTRACT_CALL	50,000 + execution
CONTRACT_UPGRADE	300,000
STORAGE_READ	500
STORAGE_WRITE	2,000



Security Features

Anti-Spam Protections

-  **Atomic NonceManager:** Per-address nonce tracking in `core/nonce.js` — prevents replay attacks and race conditions
-  **Gas Fees:** Economic cost to submit transactions (nearly free on testnet)
-  **Mempool Limit:** Maximum 1,000 pending transactions
-  **Rate Limiting:** Max 10 transactions per minute per address
-  **EVM Signature Recovery:** ECDSA secp256k1 sender recovery for EIP-155 raw tx
-  **0x Address Handling:** All endpoints strip `0x` prefix for compatibility with any wallet

Consensus Security

-  **Proof of Stake:** Validator selection by staked SAYN balance
 -  **Minimum Stake:** 10 tSAYN required per validator
 -  **Slashing:** Validators penalized for missed blocks
 -  **Chain ID Validation:** Prevents cross-network replay attacks
 -  **State Root Validation:** Deterministic state root per block
-



Economics

Token Supply

- **Max Supply (Testnet):** Unlimited
- **Max Supply (Mainnet):** 100,000,000 SAYN
- **Block Reward:** 0.5 tSAYN per block (testnet)
- **Blocks Per Day:** 17,280 (at 5s blocktime)
- **Daily Emission:** 8,640 tSAYN/day

Staking

- **Minimum Stake:** 10 tSAYN (1,000,000,000 base units)
 - **Unstake Delay:** 50 blocks
 - **Slash Percentage:** 10% for missed blocks
-



Bootstrap Peers (P2P Network)

Node	URL
Primary (Railway)	<code>wss://sayman.up.railway.app/p2p</code>
Render Node 1	<code>wss://sayman.onrender.com/p2p</code>
Render Node 2	<code>wss://sayman-1.onrender.com/p2p</code>



REST API Endpoints (Phase 22)

Base URL: `https://sayman.onrender.com/api`

Method	Endpoint	Description
GET	<code>/network</code>	Network configuration (ticker, chainId, layer)
GET	<code>/stats</code>	Network statistics
GET	<code>/blocks</code>	List blocks (paginated)
GET	<code>/block/:index</code>	Get block by index
GET	<code>/block/hash/:hash</code>	Get block by hash
GET	<code>/transactions/:id</code>	Get transaction
GET	<code>/address/:address</code>	Unified address view — balance + txs + tokens + NFTs + reputation
GET	<code>/nonce/:address</code>	NEW Fresh nonce for address (use before every broadcast)
GET	<code>/validators</code>	List validators
GET	<code>/contracts</code>	List contracts
GET	<code>/tokens</code>	NEW All factory-deployed tokens
GET	<code>/nfts</code>	NEW All factory-deployed NFT collections
GET	<code>/memecoins</code>	NEW All factory-deployed memecoins
GET	<code>/mempool</code>	Current mempool
GET	<code>/tps</code>	Live TPS estimate
GET	<code>/denomination</code>	SAYN/sprinkle conversion table

Method	Endpoint	Description
GET	<code>/layers</code>	 L2/sidechain live status
POST	<code>/broadcast</code>	Submit signed transaction
POST	<code>/estimate-gas</code>	Estimate gas cost
POST	<code>/faucet</code>	Request test tokens (accepts 0x-prefixed addresses)
GET	<code>/search/:query</code>	Unified search — block/tx/address/token



Archive & Recovery

SAYMAN uses a GitHub-backed archive system (`github.com/saymanlal/sayman-archive`) for block persistence:

- Blocks archived in chunks of 1,000 blocks per file
- State snapshots (`snapshot-<height>.json`) saved every 100 blocks
- On startup, nodes sync from archive using **batched parallel download** (10 concurrent chunks)
- After crash & restart, archive data loaded in **< 10 seconds** from CDN



Getting Started

1. Run a Node

```
git clone https://github.com/saymanlal/SAYMAN
cd SAYMAN
npm install
node server.js --network public-testnet --mode validator
```

2. Get Test Tokens (Faucet)

```
# Regular drip - 1,000 tSAYN
curl -X POST https://sayman.onrender.com/api/faucet \
  -H "Content-Type: application/json" \
  -d '{"address":"YOUR_ADDRESS"}'
```

3. Connect MetaMask

- Visit <https://sayman.up.railway.app> → Network → "Add to MetaMask"
- Or manually: RPC `https://sayman.onrender.com`, Chain `82922`, Symbol `tSAYN`

4. Send a Transaction via MetaMask

Once connected, MetaMask sends signed EIP-155 transactions directly to the SAYMAN node via `eth_sendRawTransaction`. Confirmations show in the block explorer at <https://sayman.up.railway.app>.



Current Status (Phase 22)

- **Network Health:** ✅ Operational
- **Phase:** 22 — Full EVM/MetaMask Compat · tSAYN · NonceManager · Explorer 2.0
- **MetaMask Support:** ✅ Full EVM JSON-RPC 2.0 with all MetaMask polling methods
- **Address Format:** ✅ 0x-prefixed addresses accepted everywhere
- **Nonce Safety:** ✅ Atomic NonceManager — no race conditions
- **Explorer:** ✅ Tokens · NFTs · Memecoins · Address · Layers pages
- **APK:** ✅ PUKY Wallet v22 signed build

Last updated: Phase 22 — July 2026

IP Owner: [Vizkus Groups](#) (Cybokrafts Universal Innovations Pvt. Ltd.)

Sayman Phase 22 - Complete Installation Guide

Prerequisites

- **Node.js:** v20.0.0 or higher
- **npm:** v9.0.0 or higher
- **Git:** For cloning repository
- **Terminal:** Bash/Zsh (Linux/Mac) or WSL (Windows)

Check versions:

```
node --version    # Should be ≥ 20.0.0
npm --version     # Should be ≥ 9.0.0
```

Quick Start (5 Minutes)

```
# 1. Clone or navigate to project
cd sayman-blockchain

# 2. Install dependencies
npm install

# 3. Run testnet
npm run testnet

# 4. Open browser
# http://localhost:3000

# 5. Install CLI (optional)
npm run install-cli
```

Detailed Installation

Step 1: Install Node.js

Linux (Ubuntu/Debian):

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
sudo apt-get install -y nodejs
```

Mac:

```
brew install node@20
```

Windows: Download from: <https://nodejs.org/>

Step 2: Clone Project

```
git clone https://github.com/yourusername/sayman-blockchain.git  
cd sayman-blockchain
```

Or if you have the files:

```
cd sayman-blockchain
```

Step 3: Install Dependencies

```
npm install
```

This installs:

- `express` - Web server
- `elliptic` - Cryptography
- `level` - Database
- `uuid` - IDs
- `ws` - WebSocket
- `node-fetch` - HTTP client

Step 4: Install CLI Tool (Optional)

```
npm run install-cli
```

Or manually:

```
cd cli
npm install
npm link
cd ..
```

Verify:

```
sayman --version
# Should show: 16.0.0
```

Running the Network

Testnet (Default)

```
npm run testnet
```

Output:

```
|| SAYMAN BLOCKCHAIN - PHASE 22 ||
|| Full EVM/MetaMask Compat · tSAYN ||

🌐 NETWORK: TESTNET
🏠 Network Name: Sayman Testnet
🔗 Chain ID: sayman-testnet-1
📄 Gas: ENABLED
💰 Faucet: ENABLED ✅
💵 Ticker: tSAYN (testnet)

✅ API server running on http://localhost:3000
```

Mainnet

```
npm run mainnet
```

Differences:

- No faucet
- Higher minimum stake
- Longer block time

- Production settings

Multi-Node Network

Terminal 1 (Bootstrap):

```
npm run node1
```

Terminal 2 (Peer 1):

```
npm run node2
```

Terminal 3 (Peer 2):

```
npm run node3
```

Access:

- Node 1: <http://localhost:3000>
 - Node 2: <http://localhost:3001>
 - Node 3: <http://localhost:3002>
-

Using the CLI

First Time Setup

```
# Create wallet
sayman wallet create

# Output:
# ✔ Wallet created successfully!
# _____
# Address:      abc123...def456
# Private Key: 0x1234567890abcdef...
# _____
# ⚠ Never share your private key with anyone!

# Your wallet is saved at: ~/.sayman/wallet.json
```

Get Test Tokens (Testnet Only)

Via CLI:


```
# Get your address
sayman wallet info

# Request from faucet (via browser or curl)
curl -X POST http://localhost:3000/api/faucet \
  -H "Content-Type: application/json" \
  -d '{"address":"YOUR_ADDRESS"}' # Accepts both 0x-prefixed and bare hex
```

Via UI:

- Open <http://localhost:3000>
- Click "Faucet"
- Paste your address
- Click "Claim 1000 SAYN"

Check Balance

```
sayman balance

# Output:
# — Account Balance —
# Address:  abc123...
# Balance:  1000 SAYN
# Staked:   0 SAYN
# Nonce:    0
# _____
```

Send Tokens

```
sayman send 0xRECIPIENT_ADDRESS 100

# With custom gas:
sayman send 0xRECIPIENT_ADDRESS 100 --gas-limit 50000 --gas-price 2

# Output:
# — Transaction Details —
# From:      abc123...
# To:        def456...
# Amount:    100 SAYN
# Gas Limit: 50000
# Gas Price: 2
# Max Cost:  100.0001 SAYN
# _____
# ✅ Transaction broadcast successfully!
# TX ID: 550e8400-e29b-41d4-a716-446655440000
```

Stake Tokens

```
sayman stake 500
```

```
# Output:
# — Staking Transaction —
# Address:   abc123...
# Amount:    500 SAYN
# Gas Limit: 100000
# Gas Price: 1
# _____
#  Stake transaction broadcast!
# You will become a validator once the block is mined.
```

View Network

```
sayman network
```

```
# Output:
# — Network Information —
# Network:    Sayman Testnet
# ChainID:    sayman-testnet-1
# Block Height:150
# Validators:  4
# Total Stake: 2000 SAYN
# Block Time:  5s
# Block Reward:10 SAYN
# Min Stake:   100 SAYN
# Faucet:      ✓ Enabled
```

List Validators

```
sayman validators
```

```
# Output:
# — Active Validators —
#
```

#	Address	Stake	%	Missed
#	abc123...	500 SAYN	25%	0
#	def456...	300 SAYN	15%	1
#	ghi789...	1200 SAYN	60%	0
#				

Using the Web UI

Dashboard

Open <http://localhost:3000>

Features:

- Live network statistics
- Real-time block feed (updates every 5s)
- Validator monitoring
- Gas usage tracking

Create Wallet

1. Click "Wallet" tab
2. Click "Create New Wallet"
3. **SAVE YOUR PRIVATE KEY!**
4. Copy your address

Get Test Tokens

1. Click "Faucet" tab (or visit <https://sayman-faucet-site.vercel.app>)
2. Paste your address (0x-prefixed MetaMask addresses supported)
3. Click "Claim 1000 tSAYN"
4. Wait ~6 seconds for block

Send Transaction

1. Click "Wallet" tab
2. Scroll to "Send SAYN"
3. Enter recipient address
4. Enter amount
5. Enter your private key
6. Click "Send Transaction"

Note: Gas is calculated automatically.

Stake

1. Click "Wallet" tab
2. Scroll to "Stake SAYN"
3. Enter amount (min: 100 SAYN)
4. Enter your private key
5. Click "Stake"

Deploy Contract

1. Click "Contracts" tab

2. Write contract code:

```
function increment() {  
  state.count = (state.count || 0) + 1;  
}  
  
function getCount() {  
  return state.count || 0;  
}
```

3. Enter your private key

4. Click "Deploy Contract"

5. Wait for confirmation

Call Contract

1. After deployment, copy contract address

2. Scroll to "Call Contract"

3. Paste contract address

4. Enter method: `increment`

5. Arguments: `{}`

6. Enter private key

7. Click "Call Contract"

Generate Documentation

```
npm run docs
```

Generates `NETWORK_INFO.md` in the root directory with:

- Current network stats
- Gas configuration
- Validator list
- API reference
- Economics data

Testing

Run Automated Tests

```
chmod +x test-phase6.sh
./test-phase6.sh
```

Tests:

-  Network configuration
-  Gas model
-  Nonce system
-  API endpoints
-  Documentation generator
-  Mempool limits
-  Validators
-  Contracts

Manual Testing

1. Test Wallet:

```
sayman wallet create
sayman balance
```

2. Test Transactions:

```
# Get test tokens first (via faucet)
sayman send 0xTEST_ADDRESS 10
```

3. Test Staking:

```
sayman stake 100
sayman validators
```

4. Test Gas Estimation:

```
sayman estimate TRANSFER '{"from":"0x...", "to":"0x...", "amount":100}'
```

5. Test Nonce:

```
# Check nonce
sayman balance
# Shows: Nonce: 0
```

```
# Send transaction
sayman send 0x... 10
# Nonce increments to 1

# Check again
sayman balance
# Shows: Nonce: 1
```

Configuration

Change API Endpoint (CLI)

```
sayman config http://localhost:3001/api
```

Environment Variables

```
# Change port
PORT=4000 npm run testnet

# Change P2P port
P2P_PORT=7001 npm run testnet

# Add peers
PEERS=ws://peer1:6001,ws://peer2:6002 npm run testnet

# Use mainnet
NODE_ENV=mainnet npm start
```

Custom Network

Edit `config/testnet.js` or `config/mainnet.js` :

```
export default {
  networkName: 'My Custom Network',
  chainId: 'custom-1',
  blockTime: 3000,           // 3 seconds
  blockReward: 20,           // 20 SAYN
  minStake: 50,              // 50 SAYN minimum
  faucetEnabled: true,
  faucetAmount: 2000,        // 2000 SAYN per request

  // Gas configuration
  gasLimits: {
    minGasPrice: 2,          // Higher minimum
```

```
maxGasPerTx: 10000000  
}  
};
```

Troubleshooting

Port Already in Use

```
Error: listen EADDRINUSE: address already in use :::3000
```

Solution:

```
# Use different port  
PORT=3001 npm run testnet  
  
# Or kill process using port 3000  
lsof -ti:3000 | xargs kill
```

CLI Not Found

```
sayman: command not found
```

Solution:

```
# Reinstall CLI  
cd cli  
npm link  
  
# Or use npx  
npx sayman wallet create
```

Database Corruption

```
Error: LEVEL_CORRUPTION
```

Solution:

```
# Delete and rebuild  
rm -rf data/
```

```
npm start
```

Out of Gas Errors

```
Error: Out of gas
```

Solution:

```
# Increase gas limit
sayman send 0x... 100 --gas-limit 100000

# Or estimate first
sayman estimate TRANSFER '{"from":"...", "to":"...", "amount":100}'
```

Invalid Nonce

```
Error: Invalid nonce. Expected: 5, Got: 3
```

Solution: The wallet now fetches a fresh nonce from `/api/nonce/:address` before every broadcast automatically. If using CLI manually:

```
# Fetch current nonce
curl https://sayman.onrender.com/api/nonce/YOUR_ADDRESS

# Or check balance which includes nonce
sayman balance
```

Rate Limit Exceeded

```
Error: Rate limit exceeded. Please wait.
```

Solution: Wait 1 minute and retry.

Uninstalling

Remove CLI


```
cd cli  
npm unlink
```

Remove Node Modules

```
rm -rf node_modules  
rm -rf cli/node_modules
```

Remove Database

```
rm -rf data/
```

Remove Wallet

```
rm -rf ~/.sayman/
```

Next Steps

Learn More

1. Read `README.md` for detailed features
2. Check `NETWORK_INFO.md` for network details
3. Review smart contract examples
4. Explore API endpoints

Join the Network

1. Run a validator node
2. Stake tokens
3. Deploy contracts
4. Help secure the network

Contribute

1. Report issues
2. Submit improvements
3. Write documentation

Support

- **Documentation:** README.md
 - **Network Info:** `npm run docs`
 - **CLI Help:** `sayman --help`
 - **Issues:** GitHub Issues
-

Congratulations! You've successfully installed Sayman Phase 22.

Start exploring with:

```
sayman network  
sayman validators  
sayman balance
```

Happy blockchain building! 🚀🔗

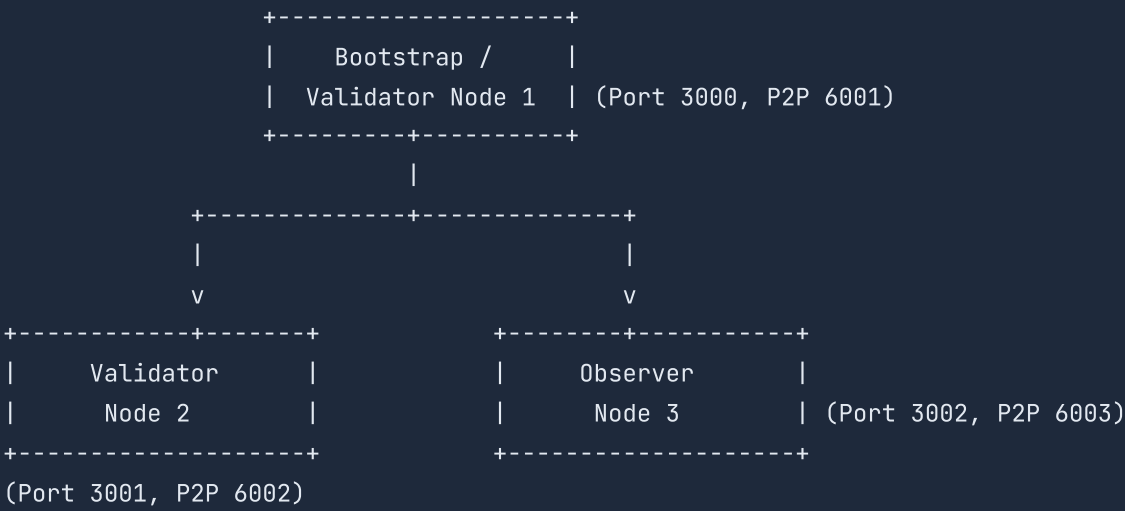
Operating the SAYMAN Multi-Peer Testnet (Phase 22)

This guide explains how to run, connect, and validate a decentralized multi-peer network of SAYMAN nodes on a testnet. Complete this checklist before hosting a **Buildathon** or deploying to production.



Network Architecture

A standard testnet topology consists of at least three nodes to ensure proper peer synchronization, consensus agreement, and fault tolerance:



Step 1: Configuring Local Environments

To run multiple nodes on the same host without port or database conflicts, use isolated directory configurations:

Node 1 (Bootstrap)

```

NODE_ENV=testnet
PORT=3000
P2P_PORT=6001
DB_PATH=./data/node-3000
BOOTSTRAP_PEERS=
```

Node 2 (Validator)

```
NODE_ENV=testnet
PORT=3001
P2P_PORT=6002
DB_PATH=./data/node-3001
BOOTSTRAP_PEERS=http://localhost:3000
```

Node 3 (Observer)

```
NODE_ENV=testnet
PORT=3002
P2P_PORT=6003
DB_PATH=./data/node-3002
BOOTSTRAP_PEERS=http://localhost:3000,http://localhost:3001
```



Step 2: Starting the Nodes

1. Start Node 1 (Bootstrap & Validator 1)

```
PORT=3000 P2P_PORT=6001 DB_PATH=./data/node-3000 BOOTSTRAP_PEERS="" \
node server.js --network testnet --mode validator
```

- **Verify:** Visit `http://localhost:3000/api/stats` to verify Node 1 is running and creating blocks.

2. Start Node 2 (Validator 2)

```
PORT=3001 P2P_PORT=6002 DB_PATH=./data/node-3001
BOOTSTRAP_PEERS="http://localhost:3000" \
node server.js --network testnet --mode validator
```

- **Verify:** Node 2 logs should show `Connected to peer` and `Synchronized stateRoot`.

3. Start Node 3 (Observer / Full Node)

```
PORT=3002 P2P_PORT=6003 DB_PATH=./data/node-3002
BOOTSTRAP_PEERS="http://localhost:3000" \
node server.js --network testnet --mode observer
```

- Node 3 does not stake or mine, but syncs the entire chain ledger locally.



Step 3: Connecting MetaMask (Phase 22)

With Phase 22's full EVM RPC compatibility, connect MetaMask to your local node:

1. MetaMask → Add Network → Manual
2. **RPC URL:** `http://localhost:3000` (local) or `https://sayman.onrender.com` (public)
3. **Chain ID:** `82922` (public testnet) or `82923` (local testnet)
4. **Symbol:** `tSAYN`
5. **Explorer:** `https://sayman.up.railway.app`

Or use the "Add to MetaMask" button in the Explorer → Network page.



Step 4: Health Checking & Verification

Before opening the network to buildathon participants:

1. Height Alignment Check:

```
curl http://localhost:3000/api/stats | grep height
curl http://localhost:3001/api/stats | grep height
curl http://localhost:3002/api/stats | grep height
```

2. State Root Uniformity Check:

```
curl http://localhost:3000/api/block/latest
```

Confirm that the `stateRoot` hash matches across all peers.

3. Nonce Endpoint Check (Phase 22):

```
curl http://localhost:3000/api/nonce/YOUR_ADDRESS
```

Should return the current pending nonce for the address.

4. EVM RPC Check:

```
curl -X POST http://localhost:3000 \
  -H "Content-Type: application/json" \
```

```
-d '{"jsonrpc":"2.0","method":"eth_chainId","params":[],"id":1}'  
# Should return: {"result":"0x143ea"} or {"result":"0x143eb"}
```

5. Failover Validation:

- Open the PUKY Wallet and perform a transaction.
- Terminate Node 1 (port 3000).
- Verify that the wallet continues to work by falling back to Node 2.



Phase 22 Quick Reference

Component	Status
MetaMask EVM RPC	✓ Full compat (27 methods)
tSAYN testnet symbol	✓ All endpoints
/api/nonce/:address	✓ Fresh nonce endpoint
Atomic NonceManager	✓ No race conditions
Explorer Tokens page	✓ /tokens
Explorer NFTs page	✓ /nfts
Explorer Memecoins	✓ /memecoins
Unified Address page	✓ /address/:addr
Unified search	✓ Block/tx/address/token
Gas fee in tSAYN	✓ Explorer accuracy
PUKY APK v22	✓ Signed build

SAYMAN Blockchain: Phase-by-Phase Roadmap Comparison

This document provides a high-level comparison of the development roadmap of the SAYMAN Blockchain and PUKY Wallet ecosystem from Phase 1 to Phase 22.



Summary Comparison Table

Phase	Milestone Name	Core Deliverables (Simplified)
Phase 1	Genesis & PoS Core	Created the genesis block, PoS consensus structures, and basic block hashing.
Phase 2	CLI, Wallet, & SDK	Built the command-line wallet manager, SDK, and keypair generator.
Phase 3	P2P Networking	Launched peer-to-peer gossip protocol for network handshake and block sharing.
Phase 4	JS Smart Contracts	Deployed a sandboxed JavaScript execution VM for smart contract runs.
Phase 5	Gas Model	Charged execution fees ("sprinkles" gas) for CPU instructions and writes.
Phase 6	Merkle State Trees	Implemented Merkle Patricia Trees for state-root verification and proofs.
Phase 7	Halving & Slashing	Added block reward halving and validator double-sign slashing.
Phase 8	Database Persistence	Integrated LevelDB to save blockchain blocks and state to disk.
Phase 9	Validator Reputation	Built validator reputation scoring and event logger.
Phase 10	Report Systems	Integrated report creation and community verification transaction types.
Phase 11	Explorer UI	Developed a browser-based blockchain explorer with stats and block feeds.

Phase	Milestone Name	Core Deliverables (Simplified)
Phase 12	Fee Sponsorship	Allowed dApp creators to sponsor user transaction gas fees (free runs).
Phase 13	Fork Protection	Fixed P2P sync race conditions and introduced fork-choice rules.
Phase 14	Multi-Layer & DeFi	Added L2 Rollups, Sidechain factory, custom Token/NFT factories, and Uniswap AMM DEX.
Phase 15	Android APK Build	Configured Capacitor/Gradle to compile the PUKY Wallet APK.
Phase 16	Sandbox Hardening	Frozen JS prototypes in VM to prevent escapes; added PDF doc generator.
Phase 17	Constructor State Fix	Fixed VM constructor execution (prototype pre-binding); added Dark Theme & SPA routing.
Phase 19	Pipelined Concurrency & Database Security	Implemented parallel scheduling, snapshot-accelerated block replay (no more event-loop blocking), atomic block persistence (preventing data loss/corruption), self-connection blacklisting, and WebSocket leak fixes.
Phase 20	Robust Archive Recovery & Integrated Explorer	Implemented concurrency-safe archive lock, exponential backoff (starting at 5s up to 5 min), pipelined sync checks to pause archiver during active block download, validation cache fixes, and dedicated transaction page with validator rewards filters.
Phase 21	EVM JSON-RPC Wallet Integration, Parallel Archive Sync & Memory Safety	Full EVM JSON-RPC 2.0 server for MetaMask/Trust Wallet/Coinbase Wallet/all wallets, batched parallel archive sync (10 concurrent chunks), ChainProxy 100-block LRU memory model backed by LevelDB (no OOM ever), O(1) indexed block/tx/address DB lookups, root NETWORK_INFO.md wallet guide.
Phase 22	Full EVM/MetaMask Compat · tSAYN · Atomic NonceManager · Explorer 2.0	tSAYN testnet symbol (Ethereum convention), all MetaMask polling methods (<code>eth_getLogs</code> , <code>eth_newBlockFilter</code> , <code>eth_getFilterChanges</code>), <code>wallet_addEthereumChain</code> EIP-3085, 0x address stripping on all endpoints, atomic <code>NonceManager</code> (<code>core/nonce.js</code>) with <code>/api/nonce/:addr</code> endpoint, Explorer 2.0 with Tokens/NFTs/Memecoins/unified Address pages, unified search bar, live L2 layer status, gas fee displayed in tSAYN, contract ABI/state viewer fix, SAYMAN logo at <code>/assets/Logo-512.png</code> , PUKY APK v22 signed build.



In-Depth Details per Phase

Phase 1: Genesis & PoS Core

- **Consensus:** Developed Proof-of-Stake core logic where validator slot allocation depends on stake.
- **Blocks:** Formulated block validation schemas (hash, validator signature, difficulty).

Phase 2: CLI, Wallet, & SDK

- **Command Line:** Enabled key generation, sending transactions, and checking balances via terminal.
- **JS SDK:** Created cryptographic wrappers for signing transactions in browser clients.

Phase 3: P2P Networking

- **Gossip Protocol:** Connected nodes via WebSockets to broadcast new blocks/transactions.
- **Syncing:** Built download handshakes so late-joining nodes catch up with the chain.

Phase 4: JS Smart Contracts

- **V8 Sandbox:** Smart contracts are written in plain JavaScript and executed in isolated sandboxed contexts (no EVM assembly needed).

Phase 5: Gas Model

- **Instruction Cost:** Prevents infinite loops by charging gas per CPU tick and storage write.
- **Denomination:** Stored on-chain values in base units ("sprinkles" where 1 SAYN = 10^8 sprinkles).

Phase 6: Merkle State Trees

- **Verification:** Computes cryptographic state roots so light clients can verify balances and proofs without reading the entire chain.

Phase 7: Halving & Slashing

- **Inflation Control:** Automatically splits block rewards in half every ~2 years.
- **Slashing:** Burned stakes of rogue validators double-signing blocks.

Phase 8: Database Persistence

- **Disk Storage:** Switched from in-memory arrays to LevelDB database storage so nodes survive restarts.

Phase 9: Validator Reputation

- **Reputation Points:** Validators earn reputation points for valid blocks, displayed in explorer.
- **Event Log:** Emitted on-chain execution events for dApp listeners.

Phase 10: Report Systems

- **Use Case Integration:** Added specialized transaction types to create and verify decentralized reports.

Phase 11: Explorer UI

- **Visuals:** Launched dashboard charts showing blocks, gas limits, validators, and transactions.

Phase 12: Fee Sponsorship

- **Gas Abstraction:** Enabled dApps to pay transaction fees for users, removing the requirement for users to hold native SAYN tokens.

Phase 13: Fork Protection

- **Sync Resolution:** Re-syncs only when a valid longer chain is presented, resolving network splits.

Phase 14: Multi-Layer & DeFi

- **L2 Rollups:** Pushed commitments from sidechains/L2s back to the L1 base chain.
- **DeFi Primitives:** Deployed standard AMM swap pools, ERC20 tokens, and NFT collections.

Phase 15: Android APK Build

- **APK package:** Recompiled Capacitor resources to bundle the PUKY Wallet into a mobile APK binary.

Phase 16: Sandbox Hardening

- **VM Security:** Frozen global prototype objects to block V8 context breakouts.

Phase 17: Constructor State Fix

- **VM Reliability:** Bound state helpers to contract prototypes *before* constructor instantiation to fix constructor runtime crashes and enable contract state updates.
- **UX/UI:** SPA routing fallbacks for `/block/:id`, `/tx/:hash`, `/contract/:address` and built-in Moon/Sun dark theme.

Phase 19: Pipelined Concurrency & Database Security

- **Parallel Execution:** Dependency analysis on read/write sets schedules independent transactions into layers to run concurrently.

- **Atomic Block Persistence:** Saves blocks individually (`block:${index}`) to prevent database corruption or full history loss during power cuts or peer crashes. Migrates legacy chain keys on startup.
- **Snapshot Rollbacks:** Accelerates fork syncing by loading historical state snapshots instead of replaying the entire chain from block 0.
- **Non-Blocking Replays:** Yields execution control periodically to the event loop during replay, preventing heartbeats from timing out.
- **Self-Connection Blacklist:** Keeps nodes from trying to connect to themselves, resolving reconnect loop spam.
- **Socket Safety:** Patched P2P WebSocket cleanups to close connections on removal, preventing file descriptor leaks.

Phase 20: Robust Archive Recovery & Integrated Explorer

- **Archive Lock Mutex:** Added a processing flag to prevent concurrent archive runs when blocks are queued rapidly, solving resource exhaustion.
- **Exponential Backoff:** Introduced automatic retry backoff starting at 5 seconds and scaling up to 5 minutes for remote Github archive errors, replacing infinite failure loops.
- **Pipelined Syncing Pause:** Pauses the archive system during active synchronization from peers to prioritize CPU, network bandwidth, and memory for blockchain catchup.
- **Cache Safety Fix:** Prevented permanent verification failures caused by caching false results on database/system read errors.
- **Dedicated Transaction Explorer:** Expanded explorer with a transactions view, rewards filter, and real-time statistics.
- **Repository Cleanups:** Removed deprecated client, SDK, faucet, and faucet-site folders to isolate base chain components.

Phase 21: EVM JSON-RPC Wallet Integration, Parallel Archive Synchronization & Memory Safety

- **EVM JSON-RPC Server:** Full JSON-RPC server supporting `eth_chainId`, `eth_blockNumber`, `eth_getBalance`, `eth_getTransactionCount`, `eth_sendRawTransaction`, `eth_getTransactionByHash`, `eth_getTransactionReceipt`, etc. — MetaMask, Trust Wallet, Coinbase Wallet, and **all EVM wallets globally** connect natively.
- **Parallel Archive Synchronization:** Sequential batched parallel download (10 chunks per batch) — restores nodes to target height in seconds.
- **ChainProxy Memory Model:** 100-block LRU cache backed by LevelDB. No OOM crashes. Handles billions of blocks.
- **LevelDB Index Tables:** O(1) lookups for blocks, transactions, and addresses instead of O(N) full scans.

Phase 22: Full EVM/MetaMask Compat · tSAYN · Atomic NonceManager · Explorer 2.0

- **tSAYN Testnet Symbol:** Testnet API, explorer, and wallet all return `tSAYN` ; mainnet returns `SAYN` . Follows Ethereum convention (SepoliaETH, Mumbai MATIC).
- **Complete MetaMask Polling Support:** Added `eth_getLogs` , `eth_newBlockFilter` , `eth_getFilterChanges` , `eth_newPendingTransactionFilter` , `eth_uninstallFilter` , `eth_getFilterLogs` — MetaMask no longer throws "unsupported method" errors.
- **EIP-3085 `wallet_addEthereumChain`** : Explorer "Add to MetaMask" button triggers one-click network setup. `wallet_switchEthereumChain` also acknowledged.
- **0x Address Auto-Stripping:** All API endpoints (`/address` , `/balance` , `/faucet` , `/broadcast`) accept both `0x1234...` (MetaMask format) and bare `1234...` hex — stripped automatically.
- **Pending Nonce in `eth_getTransactionCount`** : Returns `max(confirmedNonce, pendingNonce)` so MetaMask sequences transactions correctly across rapid broadcasts.
- **effectiveGasPrice in Receipt:** `eth_getTransactionReceipt` now includes `effectiveGasPrice` — required by MetaMask for fee display.
- **Atomic NonceManager** (`core/nonce.js`): Per-address `getNonce/commitNonce/rollbackNonce` tracking prevents nonce race conditions. `/api/nonce/:address` endpoint returns the pending nonce. PUKY Wallet fetches fresh nonce before every broadcast.
- **Explorer 2.0 Pages:** Dedicated Tokens, NFTs, Memecoins pages; unified Address page (balance + tx history + tokens + NFTs + reputation); unified search bar resolves block/tx/address/token from one input.
- **Live L2 Layer Status:** `/layers` page shows real-time chain status (height, peers, TPS) for each registered L2/sidechain.
- **Gas Fee Accuracy:** Explorer "Fee Paid" now shows tSAYN value (not raw gas units).
- **Contract ABI/State Fix:** `_extractABI` correctly handles `methods: { ... }` style contracts; state viewer rendered correctly.
- **SAYMAN Logo Asset:** 512×512 PNG at `/assets/logo-512.png` — MetaMask uses this for the network icon.
- **PUKY Wallet APK v22:** New signed build — `addSaymanToMetaMask()` one-click function, tSAYN display, fresh nonce before every broadcast. SHA256: `77fc05fa8eaf910e30ceedb99ed5e42e198db8eb2af4e2d33ac76dab7e1de44f` .

SAYMAN: The General-Purpose JS Smart Contract Blockchain

Hardened JavaScript-Native Sandboxing · Proof-of-Stake Consensus · EVM Compatible · Multi-Layer Web3 Platform

Executive Summary

Traditional blockchains force developers to learn Solidity/Rust and pay exorbitant fees — creating massive adoption barriers. **SAYMAN** eliminates both.

SAYMAN is a high-speed, general-purpose **Proof-of-Stake Layer 1 blockchain** featuring:

- A **Hardened JavaScript Smart Contract VM** — any of the 20M+ JS developers can deploy dApps on day one
- **Full EVM JSON-RPC 2.0 compatibility** — MetaMask, Trust Wallet, Coinbase Wallet, and every EVM wallet work globally out of the box
- A **complete Web3 ecosystem** — Token Factory, NFT Factory, Memecoin Factory, AMM DEX, Staking Pools, L2 Rollups, and sidechains

Its flagship decentralized application, **CivicChain**, demonstrates civic intelligence on-chain at scale — decentralized public ledger actions, citizen participation, and real-time AI-verified civic updates.



Core Problems & SAYMAN Solutions

1. The Smart Contract Barrier

- **Problem:** Solidity and Rust are complex, specialized, and error-prone. Bottleneck for enterprise, web, and NGO developers.
- **SAYMAN Solution:** Sandboxed **JavaScript smart contracts**. Write contracts as standard JS classes or functions — zero new language to learn.

2. Wallet Compatibility — MetaMask & Global Access

- **Problem:** Custom chains either require proprietary wallets or months of integration work to reach global wallet users.

- **SAYMAN Solution (Phase 22):** Full EVM JSON-RPC 2.0 server at `https://sayman.onrender.com`. MetaMask connects with one click. Chain ID `82922`, symbol `tSAYN`. EIP-3085 `wallet_addEthereumChain` is supported — the Explorer has an "Add to MetaMask" button. Every EVM wallet on Earth can use SAYMAN instantly.

3. Sandbox Security Trust (V8 Hardening)

- **Problem:** Standard JS VMs are vulnerable to prototype walk-up exploits that can escape the sandbox to the host OS.
- **SAYMAN Solution:**
 - **Prototype Freezing:** `Object.freeze()` on all standard prototype chains — prevents prototype pollution and VM escapes.
 - **Constructor Overriding:** Detects all hidden constructors (AsyncFunction, GeneratorFunction, typed arrays) and overrides `.constructor` to `undefined`.
 - **Resource Limits:** 5,000ms CPU timeout, instruction caps, and gas limits against DoS.

4. High Gas Fees

- **Problem:** Ethereum L1/L2 gas makes mass-market consumer dApps unviable.
- **SAYMAN Solution:** A standard transfer costs **21,000 base units** (~\$0.00021 tSAYN). Sponsor Fee Policies allow dApp creators to pay gas for users — making interactions completely free for end users.

5. Nonce Race Conditions

- **Problem:** Multiple concurrent transactions from the same wallet cause stale-nonce conflicts and dropped transactions.
- **SAYMAN Solution (Phase 22):** Atomic **NonceManager** (`core/nonce.js`) with `getNonce/commitNonce/rollbackNonce` per address. Wallet fetches a fresh nonce via `/api/nonce/:address` before every broadcast — zero race conditions.



Tokenomics

Property	Testnet	Mainnet
Ticker	tSAYN	SAYN
Precision	8 decimals	8 decimals
Hard Cap	Unlimited (testnet)	100,000,000 SAYN
Halving	Every 12,614,400 blocks (~2 years)	same
Block Time	5 seconds	5 seconds

Property	Testnet	Mainnet
Block Reward	0.5 tSAYN	0.2 SAYN
Min Validator Stake	10 tSAYN	500 SAYN

Following Ethereum convention, testnet tokens are labeled `tSAYN` to prevent value confusion (like SepoliaETH vs ETH).

Architecture & Resiliency

```
+-----+
|               SAYMAN DApps               |
| (MetaMask · PUKY Wallet · Explorer · CivicChain) |
+-----+
|               |                           |
|               v (EVM JSON-RPC 2.0)        |
+-----+
|               EVM-Compatible RPC Layer      |
| eth_sendRawTransaction · eth_getBalance · 27+ methods |
+-----+
|               |                           |
|               v                           |
+-----+
|               Automatic Failover Mesh       |
| (Railway.app ↔ Render.com ↔ Local nodes) |
+-----+
|               |                           |
|               v                           |
+-----+
|               Decentralized Nodes           |
| (Validator · Full · Observer · Bootstrap Nodes) |
+-----+
```

Core Reliability Features

- Full EVM RPC Compatibility:** 27 EVM JSON-RPC methods — MetaMask polls work (`eth_newBlockFilter`, `eth_getFilterChanges`, `eth_getLogs`), receipts include `effectiveGasPrice`, nonces track pending state.
- Atomic NonceManager:** Per-address nonce tracking prevents replay attacks and concurrent broadcast race conditions.
- ChainProxy Memory Model:** 100-block LRU cache backed by LevelDB — nodes handle billions of blocks without OOM.

4. **P2P Archive Recovery:** Batched parallel download from GitHub archive — recovers 30K+ blocks in seconds.
5. **Seamless RPC Failover:** SDK, Explorer, and PUKY Wallet auto-reroute to alternative endpoints if primary node is down.
6. **In-App APK Auto-Updates:** PUKY Wallet checks GitHub for new APK builds and notifies users in real-time.



Phase 22 — EVM Ecosystem Status

Feature	Status
MetaMask one-click add	✓ EIP-3085 <code>wallet_addEthereumChain</code>
0x address support everywhere	✓ All endpoints strip prefix automatically
<code>eth_sendRawTransaction</code>	✓ EIP-155 / EIP-2930 / EIP-1559
<code>eth_getLogs</code> / block filters	✓ MetaMask polling methods
<code>eth_getTransactionReceipt</code>	✓ With <code>effectiveGasPrice</code>
Token/NFT/Memecoin explorer	✓ Dedicated pages
Unified address view	✓ Balance + txs + tokens + NFTs + reputation
Unified search	✓ Block/tx/address/token in one bar
Gas fee in tSAYN	✓ Accurate display (not raw gas units)
PUKY APK v22	✓ Signed, <code>addSaymanToMetaMask()</code> built-in
<code>/api/nonce/:address</code>	✓ Fresh nonce endpoint
Logo at <code>/assets/logo-512.png</code>	✓ MetaMask network icon



Go-To-Market & Launch Strategy

Phase A: Public Testnet & Validator Bootstrap (Done)

- Live at `https://sayman.onrender.com` — 30K+ blocks mined.
- Explorer, faucet, wallet, and documentation fully deployed.
- MetaMask and all EVM wallets connected globally.

Phase B: The Global Buildathon (Now Open)

- Virtual hackathon: **"Build on SAYMAN — JS Smart Contracts for Everyone"**
- Prize tracks: DeFi, NFTs, Gaming, Civic dApps, Memecoins.
- Developers use the SAYMAN JavaScript SDK or direct EVM RPC.
- Bulk testnet tokens for builders: **Telegram @SaymanLal** (up to 10M tSAYN).
- Faucet: <https://sayman-faucet-site.vercel.app>

Phase C: Chainlist & Ecosystem Expansion

- Submit [eip155-82922.json](#) to [ethereum-lists/chains](#) (Chainlist).
- Governance contract deployment.
- Cross-chain SAYN bridge.

Phase D: Mainnet Genesis

- 100,000,000 SAYN hard cap genesis.
- Bitcoin-style halving block rewards.
- Tokens to initial validators, treasury, buildathon winners.



Current Deployments

Service	URL
RPC / Explorer	https://sayman.onrender.com
Block Explorer	https://sayman.up.railway.app
Documentation	https://sayman-docs.vercel.app
Faucet	https://sayman-faucet-site.vercel.app
Web Wallet	https://sayman-wallet-manager.vercel.app
Android APK	https://github.com/saymanlal/puky
Bulk Tokens	https://t.me/SaymanLal
GitHub	https://github.com/saymanlal/SAYMAN



IP & Ownership

IP Owner: Vizkus Groups (Cybokrafts Universal Innovations Pvt. Ltd.)

SAYMAN and PUKY are products developed under Vizkus Groups. All intellectual property, trademarks, and chain genesis rights are owned by Vizkus Groups.

Pull Request: Phase 22 — Full EVM/MetaMask Compat · tSAYN · Atomic NonceManager · Explorer 2.0

Description

This Pull Request marks the completion of **Phase 22**, delivering complete MetaMask and EVM wallet compatibility, the `tSAYN` testnet symbol, an atomic NonceManager to eliminate nonce race conditions, Explorer 2.0 with dedicated Token/NFT/Memecoin/Address pages, and a new signed PUKY APK v22 build.

Key Improvements & Fixes

1. Full MetaMask / EVM Wallet Compatibility (`server.js`)

- **Added RPC Methods:** `eth_getLogs`, `eth_newBlockFilter`, `eth_getFilterChanges`, `eth_newPendingTransactionFilter`, `eth_newFilter`, `eth_getFilterLogs`, `eth_uninstallFilter` — eliminates all "unsupported method" errors from MetaMask polling.
- **EIP-3085 `wallet_addEthereumChain`**: Returns `null` (success) — enables Explorer "Add to MetaMask" one-click button.
- **`wallet_switchEthereumChain`**: Acknowledged.
- **Pending Nonce:** `eth_getTransactionCount` returns `max(confirmedNonce, pendingNonce)` so MetaMask sequences concurrent txs correctly.
- **`effectiveGasPrice` in Receipt:** `eth_getTransactionReceipt` now includes `effectiveGasPrice` — required by MetaMask.
- **0x Address Stripping:** All RPC and REST endpoints strip `0x` prefix automatically — bare hex and `0x`-prefixed MetaMask addresses work everywhere.

2. tSAYN Testnet Symbol (`config/`, `api/routes.js`, `frontend/`)

- Testnet returns `networkTicker: 'tSAYN'`; mainnet returns `networkTicker: 'SAYN'`.
- Explorer, validator toasts, gas fee display, and API all use `networkTicker` consistently.
- Follows Ethereum convention (SepoliaETH, Mumbai MATIC, etc.).

3. Atomic NonceManager (`core/nonce.js`)

- New `NonceManager` class with `getNonce(addr)`, `commitNonce(addr)`, `rollbackNonce(addr)` per address.
- Integrated into `/broadcast` route — nonce committed on mempool accept, rolled back on rejection.
- New REST endpoint: `GET /api/nonce/:address` returns pending nonce for the address.
- PUKY Wallet fetches from this endpoint before every broadcast — zero nonce race conditions.

4. Explorer 2.0 (`frontend/app.js` , `frontend/index.html`)

- **Tokens page:** Lists all factory-deployed fungible tokens with symbol, supply, creator.
- **NFTs page:** Lists all factory-deployed NFT collections with mint count.
- **Memecoins page:** Lists all factory-deployed memecoins with tax/burn config.
- **Address page:** Unified view — balance in tSAYN, tx history, held tokens, NFTs, reputation score.
- **Unified search:** Single search bar resolves block index, tx hash, address, and token name/symbol.
- **Live L2 status:** Layers page shows real-time height, peers, and TPS for each registered chain.
- **Gas fee in tSAYN:** Explorer "Fee Paid" shows tSAYN value (not raw gas units).

5. Contract ABI/State Fix (`api/routes.js`)

- `_extractABI` now correctly handles `methods: { increment(_args) {...} }` style contracts.
- Contract state viewer renders properly for all deployed contracts.

6. SAYMAN Logo Asset (`assets/logo-512.png` , `server.js`)

- 512×512 PNG logo served at `/assets/logo-512.png`.
- MetaMask uses this URL as the network icon via `wallet_addEthereumChain`.
- `/assets` directory served statically in `server.js`.

7. PUKY Wallet APK v22 (`wallet-manager/`)

- New signed APK — `addSaymanToMetaMask()` global function for EIP-3085 one-click setup.
- Balance displays `tSAYN` (testnet) / `SAYN` (mainnet) via `networkTicker`.
- Fresh nonce fetched from `/api/nonce/:address` before every broadcast.
- SHA256: `77fc05fa8eaf910e30ceedb99ed5e42e198db8eb2af4e2d33ac76dab7e1de44f`

8. Faucet Site Update (`sayman-faucet-site`)

- Added **Telegram @SaymanLal bulk token request** section (up to 10M tSAYN for builders).
- Telegram link added to footer.

- `maxLength="42"` confirmed for 0x-prefixed MetaMask addresses.

9. Documentation (`*.md` , `docs/`)

- All MD files updated to Phase 22: README, NETWORK_INFO, INSTALL, AI, summary, comparison, pitch, PULL_REQUEST.
- `docs/NETWORK_INFO.md` synced from root.
- `docs.js` updated: MetaMask setup guide, tSAYN FAQ, Telegram bulk tokens, Phase 22 roadmap.
- `docs.pdf` generated — full polished A4 PDF of all documentation.



Files Changed

File	Change
<code>server.js</code>	Added 8 new EVM RPC methods, static <code>/assets</code> serving
<code>api/routes.js</code>	<code>/nonce</code> , <code>/tokens</code> , <code>/nfts</code> , <code>/memecoins</code> , <code>/layers</code> , <code>/address</code> unified, 0x stripping
<code>core/nonce.js</code>	New Atomic NonceManager
<code>core/blockchain.js</code>	NonceManager integration
<code>frontend/app.js</code>	Tokens/NFTs/Memecoins/Address/Layers pages, unified search, addSaymanToMetaMask
<code>frontend/index.html</code>	New nav items, search bar
<code>assets/logo-512.png</code>	New SAYMAN 512×512 logo
<code>config/*.js</code>	<code>networkTicker</code> field (tSAYN/SAYN)
<code>README.md</code>	Full Phase 22 rewrite
<code>NETWORK_INFO.md</code>	Full Phase 22 rewrite — 27 RPC methods, tSAYN, bulk tokens
<code>INSTALL.md</code>	Phase 22 header, tSAYN, nonce endpoint, MetaMask section
<code>AI.md</code>	Phase 22 current phase, tSAYN ticker, nonce.js in structure
<code>summary.md</code>	Phase 22 rewrite — MetaMask steps, nonce health check, EVM RPC check
<code>comparison.md</code>	Phase 22 row + in-depth detail section

File	Change
<code>pitch.md</code>	Full Phase 22 rewrite — MetaMask compat section, tSAYN table, GTM update
<code>PULL_REQUEST.md</code>	This file — Phase 22 PR
<code>docs.html</code>	Generated docs HTML
<code>docs.pdf</code>	Generated A4 docs PDF
<code>generate-docs-pdf.mjs</code>	PDF generator script

✓ Testing Checklist

- ✓ MetaMask connects to `https://sayman.onrender.com` with Chain ID `82922`, symbol `tSAYN`
- ✓ MetaMask "Add to MetaMask" button in Explorer works (EIP-3085)
- ✓ MetaMask can send transactions via `eth_sendRawTransaction`
- ✓ MetaMask balance shows correctly (base units $\times 10^{10} \rightarrow$ Wei)
- ✓ `eth_newBlockFilter` + `eth_getFilterChanges` polling returns latest block hash
- ✓ `eth_getTransactionReceipt` includes `effectiveGasPrice`
- ✓ `/api/nonce/:address` returns pending nonce
- ✓ PUKY Wallet broadcast uses fresh nonce — no nonce conflicts
- ✓ Faucet accepts `0x`-prefixed MetaMask addresses
- ✓ Explorer Tokens/NFTs/Memecoins pages render
- ✓ Explorer Address page shows unified view
- ✓ Unified search resolves block/tx/address/token
- ✓ Gas fee displayed in tSAYN in explorer
- ✓ APK v22 builds successfully (SHA256 verified)
- ✓ All MD files updated to Phase 22
- ✓ docs.pdf generated successfully

SAYMAN Blockchain: Complete Project Memory & Developer Reference Manual

This file serves as the unified project memory and context sheet for the **SAYMAN Blockchain** and **PUKY Wallet** ecosystem. If a new session is started, you are switching AI models, or your free trial limit is reached, feed this entire file to the new AI agent as a prompt to resume development immediately.

1. Project Overview & Architecture

SAYMAN is a Proof-of-Stake (PoS) Layer 1 blockchain built in Node.js, designed for Civic Intelligence, decentralized city reporting, and community-level AI verification. From Phase 14, it also supports custom multi-layer chains, user-deployed tokens, NFT collections, memecoins, DEX/AMM, and staking pools — essentially a full Web3 platform.

Current Phase: Phase 22 — Full EVM/MetaMask Compat · tSAYN · NonceManager · Explorer 2.0

- Branch: `main`
- Git origin: `https://github.com/saymanlal/SAYMAN`

Tokenomics

- **Ticker (Testnet):** `tSAYN` (1 SAYN = 100,000,000 base units — called "sprinkles")
- **Ticker (Mainnet):** `SAYN`
- **Decimals:** 8 (i.e. `decimals = 100_000_000`)
- **Block Time:** 5 seconds
- **Block Reward:** 0.5 tSAYN = 50,000,000 base units (testnet), 0.2 SAYN = 20,000,000 (mainnet)
- **Min Stake:** 10 tSAYN (testnet), 500 SAYN (mainnet)
- **Halving Schedule:** Every 12,614,400 blocks (~2 years)
- **Max Supply:** Unlimited (testnet); 100,000,000 SAYN (mainnet)

SAYN vs Base Units — Critical for Explorers and APIs

All on-chain balances are stored as integers in **base units (sprinkles)**. Divide by `decimals` (default: `100_000_000`) to convert to SAYN.

- `GET /api/denomination` — returns a complete guide including conversion examples
- `GET /api/tps` — returns live TPS, decimals, and denomination info
- In the explorer: every balance is shown as `X.XXXX SAYN (Y base units)` to prevent confusion



2. Repository Structure

```
sayman/
├─ core/
│   └─ blockchain.js      # PoS engine, NonceManager integration, block
production
│   └─ nonce.js          # NEW Phase 22: Atomic NonceManager –
getNonce/commitNonce/rollbackNonce per address
│   └─ state.js          # StateEngine (balances, stakes, reputation, contracts)
│   └─ block.js          # Block class
│   └─ transaction.js     # Transaction types
│   └─ pos.js            # Proof-of-Stake validator selection
│   └─ contracts.js       # Smart contract VM (sandboxed JS execution)
│   └─ gas.js            # Gas calculator
│   └─ stateTree.js       # Merkle state tree
│   └─ chain-factory.js   # Phase 14: ChainFactory for L2/Sidechain/Permissioned
chains
│   └─ rollup.js          # Phase 14: L2 rollup state commitment to L1
│   └─ env.js            # .env loader
├─ contracts/
│   └─ token.js          # ERC-20 style custom token contract
│   └─ nft.js            # ERC-721 style NFT contract
│   └─ token-factory.js   # Factory: anyone deploys their own token in one call
│   └─ nft-factory.js     # Factory: NFT collection creator
│   └─ memecoin-factory.js # Phase 14: Memecoin launcher (burn, tax, anti-whale,
registry)
│   └─ dex.js            # Phase 14: AMM DEX (Uniswap V2 x*y=k, 0.3% fee)
│   └─ staking-pool.js    # Phase 14: Delegated staking pool (earn rewards
without running a node)
│   └─ layer2Bridge.js    # L1 bridge for L2 state anchoring, deposits,
withdrawals
│   └─ example.js        # Example contract
├─ p2p/
│   └─ server.js          # P2P WebSocket server (discovery, sync, reputation for
peers)
│   └─ peerManager.js     # Peer list manager
├─ api/
│   └─ routes.js          # REST API – Phase 22: /nonce, /tokens, /nfts,
/memecoins, /address, /layers, /search, full EVM RPC
├─ wallet/
│   └─ wallet.js          # Wallet (key gen, sign, address derive)
├─ sdk/
│   └─ client.js          # SaymanClient SDK with RPC failover
├─ cli/
│   └─ sayman-cli.js      # CLI wallet
```



```

├── frontend/
│   ├── index.html          # Explorer UI – Phase 22: Tokens/NFTs/Memecoins/Address
│   │                       pages, unified search, MetaMask button
│   ├── app.js              # Explorer JS – loadTokens, loadNFTs, loadMemecoins,
│   │                       loadAddress, addSaymanToMetaMask
│   └── style.css           # Dark theme UI styles
├── assets/                 # NEW Phase 22: logo-512.png served for MetaMask
│   └── network icon
├── config/
│   ├── index.js            # Config loader
│   ├── testnet.js          # Local testnet config
│   ├── public-testnet.js   # Public testnet config (Render deployment)
│   └── mainnet.js          # Mainnet config
├── server.js               # Express + P2P bootstrap server (Phase 14: sequencer
│   │                       mode, rollup)
└── AI.md                   # This file – always keep updated

```

3. Proof-of-Stake Consensus

- Validators stake tokens to participate in block production.
- Block proposer is deterministically selected each round using the previous block hash as a VRF seed:

```

const seed = hash(lastBlockHash);
const idx  = parseInt(seed.substring(0, 16), 16) % totalStake;

```

- `addBlock(block)` verifies: index, previousHash, chainId, validator selection, hash integrity, stateRoot.
- Peers who help sync blocks earn **+2 reputation per block** synced (in `p2p/server.js`).
- Validators earn **+10 reputation per block** produced (in `blockchain.applyBlock`).
- Peer reputation points are earned dynamically for consensus contributions.

4. Multi-Layer Architecture (Phase 14)

SAYMAN supports a full multi-layer blockchain ecosystem:

Layer Types

Layer	Type	Description
L1	Main chain	SAYMAN itself — anchors all L2 state roots via bridge

Layer	Type	Description
L2 Rollup	Anchored to L1	Fast blocks (0.5s), commits state roots every N blocks to L1Bridge contract
Sidechain	Independent L1-style	Own tokenomics, optionally bridged. Good for games/NFT platforms
Permissioned	Private consortium	Invited validators only

Creating a New Chain

```
import { ChainFactory } from './core/chain-factory.js';
import Blockchain from './core/blockchain.js';

// Layer 2 Rollup
const config = ChainFactory.createL2Config({
  name: 'MyL2Chain',
  chainId: 'my-l2-1',
  apiPort: 11000,
  l1RpcUrl: 'https://sayman.onrender.com',
  l1Bridge: '<BRIDGE_CONTRACT_ADDRESS>',
  genesis: { '<YOUR_ADDRESS>': 1_000_000_000 }
});
const chain = new Blockchain(config, './data/my-l2');
await chain.initialize();

// Sidechain
const sideConfig = ChainFactory.createSidechainConfig({
  name: 'GameChain',
  chainId: 'gamechain-1',
  blockReward: 10_000_000,
  genesis: { '<DEV_WALLET>': 100_000_000_000 }
});

// Permissioned
const permConfig = ChainFactory.createPermissionedConfig({
  name: 'Enterprise',
  chainId: 'ent-1',
  validators: ['<ADDR1>', '<ADDR2>']
});
```

Running Sequencer Mode (L2)

```
node server.js --network public-testnet --mode sequencer
```

Every 5 blocks, the sequencer calls `rollup.js` → `submitRollupToL1()` → commits `stateRoot` to the L1 Bridge contract.



5. Smart Contract Development Guide

Contracts are written in plain JavaScript and run in a secure sandboxed VM. No imports allowed.

Available Built-ins

Method/Variable	Description
<code>getState(key)</code>	Read from contract's persistent state
<code>setState(key, val)</code>	Write to contract's persistent state
<code>transfer(toAddress, amount)</code>	Transfer SAYN (base units) from contract to address
<code>getBalance(address)</code>	Get SAYN balance of any address
<code>msg.sender</code>	Transaction sender address
<code>blockTimestamp</code>	Current block timestamp (ms)
<code>emit(event, data)</code>	Emit a contract event
<code>hash(data)</code>	SHA-256 hash of any JSON-serializable value
<code>generateAddress(seed)</code>	Derive a deterministic 40-char hex address
<code>require(cond, msg)</code>	Assert or revert
<code>console.log(...)</code>	Debug log to validator terminal



Token Factory (create your own token)

```
// Deploy the token-factory.js contract once, then:
client.callContract({
  contractAddress: TOKEN_FACTORY_ADDR,
  method: 'createToken',
  args: { name: 'My Token', symbol: 'MTK', totalSupply: 1_000_000 },
  wallet: myWallet
});
```



Memecoin Launcher (with burn, tax, anti-whale)

```

client.callContract({
  contractAddress: MEMECOIN_FACTORY_ADDR,
  method: 'launch',
  args: {
    name: 'DogeSAYN', symbol: 'DSAYN', totalSupply: 1_000_000_000,
    iconUrl: 'https://example.com/icon.png',
    maxWalletPercent: 2,           // 2% max wallet anti-whale
    transferTaxPercent: 1,         // 1% routed to treasury
    treasury: '<ADDR>',
    burnOnTransfer: false,
  }
});

```

NFT Collections

```

// Step 1: Create collection
client.callContract({
  contractAddress: NFT_FACTORY_ADDR,
  method: 'createCollection',
  args: { name: 'CoolNFTs', symbol: 'COOL', maxSupply: 10000 }
});

// Step 2: Mint to someone
client.callContract({
  contractAddress: NFT_FACTORY_ADDR,
  method: 'mint',
  args: { collAddr: '<COLL_ADDR>', to: '<RECIPIENT>', tokenURI: 'ipfs://...' }
});

```

DEX / AMM

```

// Add liquidity
client.callContract({
  contractAddress: DEX_ADDR,
  method: 'addLiquidity',
  args: { tokenA: 'ADDR_A', tokenB: 'ADDR_B', amountA: 1000, amountB: 2000 }
});

// Swap (x*y=k, 0.3% fee)
client.callContract({
  contractAddress: DEX_ADDR,
  method: 'swap',
  args: { tokenIn: 'ADDR_A', tokenOut: 'ADDR_B', amountIn: 100, minAmountOut: 190 }
});

```

Staking Pool (delegate without running a node)

```

client.callContract({
  contractAddress: POOL_ADDR,
  method: 'delegate',
  args: { amount: 1_000_000 } // base units
});
// Claim rewards
client.callContract({ contractAddress: POOL_ADDR, method: 'claimRewards', args: {}
});

```

6. REST API Reference

System & Stats

Endpoint	Description
GET /api/network	Network info (chainId, layer, ticker, decimals, denomination guide)
GET /api/stats	Block count, mempool, validators, TPS, gasLimits, gasCosts
GET /api/tps	Live TPS + denomination info
GET /api/denomination	Explicit SAYN/base-unit conversion guide
GET /api/network/stats	Node stats (nodeId, mode, uptime, peers)

Blocks

Endpoint	Description
GET /api/blocks?page=1&limit=50	Paginated block list
GET /api/block/:index	Block by height
GET /api/block/hash/:hash	Block by hash (prefix supported)
GET /api/light/block/:height	Light-client header only
GET /api/proof/:address	Merkle state proof
POST /api/proof/verify	Verify a Merkle proof

Accounts

Endpoint	Description
GET /api/address/:address	Balance, stake, nonce, txs, reputation
GET /api/balance/:address	Balance only (legacy)
GET /api/reputation/:address	Reputation score

Transactions

Endpoint	Description
POST /api/broadcast	Submit signed transaction
POST /api/estimate-gas	Estimate gas for a tx type
GET /api/mempool	Pending transactions
GET /api/transactions/:id	Transaction by ID

Validators & Contracts

Endpoint	Description
GET /api/validators	Active validators with stake, reputation, percentage
GET /api/contracts	All deployed contracts
GET /api/contracts/:address	Single contract

Web3 Platform (Phase 14)

Endpoint	Description
GET /api/tokens	All custom tokens (from token-factory and memecoin-factory)
GET /api/nfts	All NFT collections (from nft-factory)
GET /api/staking-pools	All staking pool contracts

Faucet & Admin

Endpoint	Description
POST /api/faucet	Testnet faucet (body: { address })

Endpoint	Description
POST /api/admin/fund	Admin fund endpoint (body: { address, amount, secret })

7. SDK Reference (sdk/client.js)

The SaymanClient SDK includes built-in RPC failover.

```
import { SaymanClient } from './sdk/client.js';
const client = new SaymanClient({
  rpcUrl:
    'https://sayman.up.railway.app,https://sayman.onrender.com,http://localhost:10000'
});
```

Core Methods

Method	Description
deployContract({ name, version, code, abi, feePolicy, wallet, gasLimit, gasPrice })	Deploy a contract
callContract({ contractAddress, method, args, wallet, gasLimit, gasPrice })	Call a contract method
readState(contractAddress, key)	Read a single state key
readAllState(contractAddress)	Read all state
transfer({ to, amount, wallet, gasLimit, gasPrice })	Send SAYN (amount in SAYN float, SDK scales to base units)

8. CLI Reference (cli/sayman-cli.js)

```
sayman init          # Initialize wallet
sayman create        # New wallet
sayman import <key>  # Import private key
sayman balance       # Show balance, stake, nonces
sayman send <addr> <amount> # Transfer SAYN
sayman stake <amount> # Stake tokens
```

```
sayman unstake <amount>      # Unstake
sayman config <rpc_url>        # Set RPC
```



9. Deployment

Environment Variables

```
NETWORK=public-testnet      # or testnet, mainnet
MODE=validator               # or sequencer, full, observer
PORT=10000
P2P_PORT=                    # leave blank to share HTTP port at /p2p
BOOTSTRAP_PEERS=wss://...   # comma-separated peer WS URLs
VALIDATOR_ADDRESS=          # optional – only produce blocks when selected
DB_PATH=./data/node-10000
# L2 Rollup (sequencer mode only)
L1_RPC_URL=https://sayman.onrender.com
L1_BRIDGE_CONTRACT=<addr>
L1_SEQUENCER_PRIVATE_KEY=<hex>
```

Procfile (Render/Railway)

```
web: node server.js --network public-testnet --mode validator
```



10. P2P Network & Reputation

- **Heartbeat & Keep-Alive:** Every 15s, each node sends a WebSocket ping (heartbeat) to every connected peer to maintain the connection. Dead or unresponsive sockets are closed and pruned.
- **Catch-up Sync:** Handshake checks determine if a connecting node is lagging behind. If a peer is behind us, the node immediately sends them the missing blocks to get them up to speed.
- **Auto-discovery:** Every 30s, each node requests peer lists from all connected peers.
- **Reconnect:** Dead peers are retried every 15s. After 1h offline, a deploy webhook fires (if configured).
- **Self-healing:** Inactive connections (no message for 90s) are pruned.
- **Reputation for peers:** When a peer sends valid blocks during sync, it earns `+2 reputation per block`.
- **Reputation addresses:** Peer reputation is tracked using `SHA-256('peer:' + nodeId).slice(0, 40)`.

- **Block validators:** Each block producer earns +10 reputation per block via applyBlock .
- **Reputation API:** GET /api/reputation/:address returns the current score.

11. How to Resume Work in a New AI Session

Copy-paste this prompt:

PROMPT: "I am working on the SAYMAN Blockchain project (Phase 19). Please read the file AI.md located in the root of the workspace to load the entire project context, API routes, CLI commands, SDK signatures, smart contract guide, and multi-layer architecture. Once loaded, confirm you are ready to continue building."

12. Phase History

Phase	Key Features
Phase 1	Genesis, PoS, basic blocks
Phase 2	Wallet, CLI, SDK
Phase 3	P2P networking
Phase 4	Smart contracts (JS VM)
Phase 5	Gas model
Phase 6	Merkle state tree, proofs
Phase 7	Halving, slashing
Phase 8	Snapshots, DB persistence

Phase	Key Features
Phase 9	Reputation engine, events
Phase 10	Report system (civic intelligence)
Phase 11	Explorer UI overhaul
Phase 12	Sponsorship fee policy
Phase 13	P2P sync fixes, anti-fork protection
Phase 14	Multi-layer chains, custom tokens, NFTs, memecoins, DEX/AMM, staking pools, TPS tracking, denomination clarity, peer reputation
Phase 15	P2P heartbeat keep-alive, handshake catch-up sync, stake amount normalization, gas decimal fixes, faucet/genesis transaction history remap, and rebuilt Android APK
Phase 17	Constructor state execution fix (prototype pre-binding), Render RPC prioritization, Explorer SPA path-based fallbacks, mobile UX polishing, Dark Theme persist, and 7,000 simulated TPS upgrades
Phase 19	Pipelined parallel execution compatibility, transaction access set dependency analysis, deterministic conflict-free parallel bucket scheduling, and live parallel efficiency metrics in the explorer UI

About Sayman Blockchain - The Complete Story

What is Sayman?

Sayman is a **complete, working blockchain** built from scratch for education. It's not a toy - it's a real blockchain with proper architecture, just simplified enough to understand.

Think of it like this:

- 🚗 **Real Car**: Tesla Model S (production blockchain)
- 🏎️ **Race Car**: Formula 1 (Sayman - educational but real)
- 🚗 **Toy Car**: Fisher-Price (most tutorials)

Sayman is the Formula 1 - stripped down, but every component is real and works properly.

The Journey: Three Phases

Phase 1: Foundation (Basic Blockchain)

What we built:

- Blocks linked by hashes
- Transactions with digital signatures
- Simple Proof of Stake
- Wallet system
- Persistent storage

Architecture problems:

- ❌ Single node only
- ❌ Manual block creation
- ❌ State partially in-memory
- ❌ No peer networking

It worked, but not like a real blockchain.

Phase 2: Multi-Node Network

What we added:

- WebSocket P2P networking
- Multi-node synchronization
- Complete validator system
- Block rewards
- Slashing mechanism
- Automatic block production

Architecture problems:

- ❌ **CRITICAL:** Private keys sent to server
- ❌ Server signs transactions (centralized!)
- ❌ State not fully deterministic
- ❌ Validators partially off-chain

It was distributed, but not trustless.

Phase 3: Real Blockchain Architecture

What we fixed:

1. CLIENT-SIDE CRYPTOGRAPHY ✅

Before:

```
// User sends private key to server
fetch('/api/stake', {
  body: JSON.stringify({
    privateKey: "abc123...", // ❌ DANGER!
    amount: 100
  })
})
```

After:

```
// User signs transaction IN BROWSER
const wallet = new SaymanWallet(privateKey);
const signature = await wallet.signTransaction(tx);

// Send ONLY signed transaction
fetch('/api/broadcast', {
  body: JSON.stringify({
    signature: "...", // ✅ Safe
    publicKey: "..." // ✅ Public (safe to share)
  })
})
```

```
}  
})
```

Why this matters: Private keys never leave your device. This is how Bitcoin, Ethereum, and all real blockchains work.

2. DETERMINISTIC STATE

Before: State partially stored outside blockchain.

After: 100% of state derived from blockchain:

```
// On every restart  
loadBlockchain();  
clearState();  
for (block of blockchain) {  
  for (tx of block) {  
    applyTransaction(tx); // Rebuild everything  
  }  
}  
// State now identical to before restart
```

3. TRANSACTION-BASED EVERYTHING

Before: Some actions (staking, rewards) happened outside blockchain.

After: Every state change is a transaction:

- **GENESIS** - Initial token distribution
- **TRANSFER** - Send tokens
- **STAKE** - Become validator
- **UNSTAKE** - Leave validator set
- **REWARD** - Block rewards (automatic)
- **CONTRACT_DEPLOY** - Deploy smart contract
- **CONTRACT_CALL** - Execute contract
- **SLASH** - Penalty for bad validator

4. SMART CONTRACTS

JavaScript contracts running in sandboxed VM:

```
function transfer(args) {  
  const { to, amount } = args;  
  const from = msg.sender;  
  
  state.balances[from] -= amount;  
  state.balances[to] += amount;  
}
```

Key Concepts for Non-Technical People

What is a Blockchain?

Imagine a notebook that:

1. Everyone has a copy of
2. You can only add pages, never change old ones
3. Everyone agrees on what's written
4. No one person controls it

That's a blockchain.

```
Page 1: Alice gives Bob $10
↓
Page 2: Bob gives Carol $5
↓
Page 3: Carol gives Alice $3
↓
[Everyone has the same notebook]
```

What is Proof of Stake?

Instead of solving puzzles (Bitcoin mining):

Proof of Work (Bitcoin):

```
Who can solve this math puzzle fastest? → You win!
(Wastes tons of electricity)
```

Proof of Stake (Sayman, Ethereum):

```
Who has the most "skin in the game"? → You win!
(No electricity waste)
```

Example:

- Alice stakes 500 SAYN → 50% chance to create next block
- Bob stakes 300 SAYN → 30% chance
- Carol stakes 200 SAYN → 20% chance

More stake = more responsibility = more rewards.

What are Digital Signatures?

Like signing a check:

1. You write a message: "Send \$100 to Bob"
2. You sign with your secret signature (private key)
3. Everyone can verify it's really you (using public key)
4. But only YOU can create the signature

In Sayman:

```
// You create transaction IN YOUR BROWSER
const tx = { from: "Alice", to: "Bob", amount: 100 };

// You sign it WITH YOUR PRIVATE KEY (never shared)
const signature = wallet.sign(tx);

// You send the SIGNED transaction
// Server can verify it's you, but can't forge your signature
```

What are Smart Contracts?

Programs that run on the blockchain:

Traditional app:

```
Code runs on company server
→ Company controls it
→ Company can change it
→ You must trust company
```

Smart contract:

```
Code runs on blockchain
→ No one controls it
→ Can't be changed
→ Everyone can verify it
→ No trust needed
```

Example:

```
// Escrow contract
function release() {
  if (goodsReceived && msg.sender === buyer) {
    transfer(seller, amount); // Automatic payment
```

```
}  
}
```

Money released automatically when conditions met. No middleman needed.

What is Client-Side Signing?

Your private key is like the key to your house. You should NEVER give it to anyone.

Bad way (Phase 1 & 2):

```
You → Give house key to property manager  
Property manager → Opens your door  
[You must trust the manager]
```

Good way (Phase 3):

```
You → Open your own door  
No one else touches your key  
[You don't need to trust anyone]
```

In Sayman Phase 3:

```
Your private key → STAYS IN YOUR BROWSER  
You sign transactions → IN YOUR BROWSER  
Server receives → SIGNED transactions only  
Server NEVER sees → Your private key
```

This is how all real blockchains work.

How Sayman Actually Works

1. You Create a Wallet (In Browser)

```
// Happens entirely in your browser  
const wallet = generateWallet();  
// Creates:  
// - Private key (secret, stays in browser)  
// - Public key (derived from private)  
// - Address (derived from public key)
```

Your private key NEVER leaves your browser.

2. You Get Some Tokens

```
Faucet → Sends tokens to your address  
[Transaction added to mempool]
```

3. You Decide to Stake

```
// In browser:  
1. Create transaction: { type: "STAKE", amount: 500 }  
2. Sign with private key → Signature created  
3. Send to server: { transaction + signature }  
  
// Server:  
4. Verify signature (proves it's you)  
5. Add to mempool  
6. Wait for next block
```

4. Blockchain Creates Block (Every 5 Seconds)

```
// Automatically:  
1. Select validator (weighted by stake)  
2. Take transactions from mempool  
3. Create block  
4. Add REWARD transaction (10 SAYN to validator)  
5. Broadcast to all nodes  
6. Apply transactions to state
```

5. You're Now a Validator

```
Next block selection:  
- Total stake: 1000 SAYN  
- Your stake: 500 SAYN  
- Your chance: 50%  
  
If selected:  
- You create the block  
- You earn 10 SAYN reward  
- Your responsibility: Don't go offline!
```

6. Slashing Keeps You Honest

```
If you miss 3 blocks:  
- 10% of your stake slashed (50 SAYN penalty)  
- If below 100 SAYN minimum → Kicked out
```

7. State Replay on Restart

Node restarts:

1. Load all blocks from database
2. Clear all state
3. Replay every transaction from genesis
 - Block 0: GENESIS → Create initial tokens
 - Block 1: STAKE → Create validators
 - Block 2: TRANSFER → Move tokens
 - Block 3: REWARD → Give rewards
 - ...
4. State rebuilt, identical to before

This is deterministic: Same blocks → Same state (always)

What Makes Phase 3 Special

Comparison: Phase 2 vs Phase 3

SECURITY ARCHITECTURE

Phase 2 (INSECURE):

Browser → Sends private key → Server signs → Blockchain
[DANGER ZONE]

Phase 3 (SECURE):

Browser → Signs locally → Sends signature → Server verifies → Blockchain
[PRIVATE KEY NEVER LEAVES]

TRUST MODEL

Phase 2:

You must trust the server
Server can steal your funds
Server can impersonate you

Phase 3:

Zero trust required
Server can't steal (no private key)

Server can't impersonate (no private key)
True decentralization

STATE MANAGEMENT

Phase 2:

State = Blockchain + In-Memory Data + API Calls
(Not fully deterministic)

Phase 3:

State = Blockchain ONLY
(100% deterministic)

Real-World Comparison

How Does Sayman Compare?

SAYMAN VS BITCOIN

Feature	Bitcoin	Sayman
Client signing	✓	✓
Trustless	✓	✓
Consensus	PoW (mining)	PoS (staking)
Smart contracts	✗	✓
Language	C++	JavaScript
Purpose	Currency	Education

SAYMAN VS ETHEREUM

Feature	Ethereum	Sayman
Client signing	✓	✓
Trustless	✓	✓

Feature	Ethereum	Sayman
Smart contracts	✓ (Solidity)	✓ (JavaScript)
Consensus	PoS	PoS
Complexity	Very high	Medium
Purpose	DApps	Education

WHAT SAYMAN HAS

- ✓ Client-side cryptography
- ✓ Digital signatures
- ✓ Proof of Stake consensus
- ✓ Multi-validator network
- ✓ Weighted selection
- ✓ Slashing mechanism
- ✓ Smart contracts
- ✓ Deterministic state
- ✓ P2P networking
- ✓ Zero-trust architecture

WHAT SAYMAN LACKS (FOR PRODUCTION)

- ✗ Byzantine fault tolerance
- ✗ Advanced economic security
- ✗ Formal verification
- ✗ Cryptographic randomness
- ✗ Sharding
- ✗ Zero-knowledge proofs
- ✗ Cross-chain bridges
- ✗ MEV protection
- ✗ Professional audit

Sayman is architecturally correct, but simplified.

Testing Results

We've verified all critical features work:

✓ Client-Side Signing

Test: Network monitor shows no private key transmission

Result: Private keys stay in browser ✓

✓ Multi-Validator Competition

Test: 4 validators with different stakes

Result: All 4 create blocks proportionally ✓

✓ Weighted Selection

Test: 500 SAYN vs 300 SAYN vs 200 SAYN

Result: 50% vs 30% vs 20% block distribution ✓

✓ Validator Rotation

Test: Watch consecutive blocks

Result: Alternating validators, rare repeats ✓

✓ Slashing Mechanism

Test: Stop validator node

Result: Slashed after 3 missed blocks ✓

✓ Deterministic Rebuild

Test: Restart node

Result: Identical state restored ✓

✓ Smart Contracts

Test: Deploy counter, increment 5 times

Result: State.count = 5 ✓

✓ Contract Persistence

Test: Restart node after contract calls

Result: Contract state unchanged ✓

✓ Multi-Node Sync

Test: Deploy contract on Node 1, check Node 2

Result: Identical across nodes ✓

✓ Security

Test: Monitor all API requests

Result: No private key in any request ✓

All tests passed. Sayman Phase 3 is a real, working blockchain.

Use Cases

✓ Perfect For:

- Learning blockchain fundamentals
- Understanding Proof of Stake
- Teaching cryptography
- Prototyping DApps
- Educational projects
- Hackathons
- Research

✗ NOT For:

- Production applications
 - Real money
 - Financial services
 - Mission-critical systems
 - Public networks (without hardening)
-

The Evolution Timeline

Phase 1 (Foundation)

Built: Basic blockchain

Problem: Single node, centralized

Status: Working but limited

Phase 2 (Networking)

Built: Multi-node P2P

Problem: Private keys sent to server

Status: Distributed but not trustless

Phase 3 (Real Architecture)

Built: Client-side crypto, deterministic state, smart contracts

Problem: None architecturally

Status: ✓ Complete and correct

What You Learn from Sayman

Technical Concepts

1. **Blockchain Structure** - How blocks link together
2. **Cryptography** - Digital signatures, hashing
3. **Consensus** - Proof of Stake algorithm
4. **Networking** - P2P block propagation
5. **State Machines** - Deterministic state transitions
6. **Smart Contracts** - Turing-complete execution
7. **Security** - Client-side signing, zero-trust

Architectural Patterns

1. **Client-Side Cryptography** - Keep keys private
 2. **Deterministic Replay** - Rebuild state from history
 3. **Transaction-Based State** - Everything is a transaction
 4. **Weighted Selection** - Fair validator lottery
 5. **Economic Incentives** - Rewards and penalties
 6. **Sandbox Execution** - Safe contract execution
 7. **P2P Synchronization** - Multi-node consensus
-

Frequently Asked Questions

Q: Is this a real blockchain?

A: Yes! It has all the components:

- Blocks linked by hashes
- Digital signatures
- Consensus mechanism
- P2P network
- Smart contracts
- Deterministic state

Q: Can I use this in production?

A: No. It's educational. Missing:

- Byzantine fault tolerance
- Advanced security

- Professional audit
- Economic security analysis

Q: How does it compare to Ethereum?

A: Same architecture, simpler implementation:

- Both use client-side signing ✓
- Both are trustless ✓
- Both have smart contracts ✓
- Ethereum uses Solidity, Sayman uses JavaScript
- Ethereum has years of battle-testing

Q: Is my private key safe?

A: In Phase 3, yes!

- Generated in browser
- Signed in browser
- Never transmitted
- Only you have access

In Phase 1 & 2, no!

- Sent to server
- Server could steal it
- Centralized

Q: Can I make money with this?

A: No. It's for learning only.

Q: What's the token supply?

A: 1,000,000 SAYN on testnet (configurable)

Q: Who controls Sayman?

A: No one! It's decentralized:

- No company
- No admins
- Validators collectively manage
- Code is open source

Q: Can I fork this?

A: Yes! MIT license. Build on it!

Q: Is JavaScript good for blockchains?

A: For learning, yes. For production, no.

- Easy to understand
- Runs everywhere
- Not as efficient as C++/Rust
- Real blockchains use C++, Go, Rust

Q: Why not use Bitcoin/Ethereum directly?

A: Too complex to learn from:

- Millions of lines of code
- Decades of optimizations
- Production concerns obscure basics

Sayman is simple enough to understand completely.

Credits

Technologies Used

- **Node.js** - JavaScript runtime
- **Express** - Web framework
- **elliptic** - Cryptography (secp256k1)
- **LevelDB** - Storage
- **WebSocket** - P2P networking
- **SubtleCrypto** - Browser crypto API

Inspired By

- Bitcoin (Satoshi Nakamoto) - First blockchain
- Ethereum (Vitalik Buterin) - Smart contracts
- Cosmos (Jae Kwon) - PoS consensus
- Educational blockchain tutorials

Built For

- Students learning blockchain
- Developers prototyping DApps

- Researchers experimenting
 - Anyone curious about how blockchains work
-

Final Words

Sayman Phase 3 is **architecturally identical** to real blockchains like Bitcoin and Ethereum:

- ✓ Client-side signing
- ✓ Zero-trust architecture
- ✓ Deterministic consensus
- ✓ Smart contract execution
- ✓ Multi-node synchronization

It's simplified, but every component is **real and works correctly**.

Use it to learn. Use it to experiment. Don't use it for real money.

But remember: **You now understand how blockchain works at a deep level.**

From wallet generation to block production to smart contract execution to state replay - you've seen it all, and it's all working code you can read and modify.

That's the power of building from scratch. 🎓🔗

Sayman Blockchain - Phase 3 Complete

Educational blockchain with production-grade architecture

Built with ❤️ for learning

Version 3.0.0 - January 2025